

COMP2048 – Master Revision Sheet

Contents

1. Mathematics for Computer Science	4
1.1. Proof $\sqrt{2}$ Is Irrational	4
1.2. Show that the cardinality of the Power Set $P(S)$ is larger than the cardinality of the set of natural numbers ($\mathbb{N}$)	4
1.3. Show Gödel's Incompleteness Theorem	5
1.3.1. Setup/Definitions	5
1.3.2. Proof of Gödel's Incompleteness Theorem	6
1.3.3. Proof of Gödel's Second Incompleteness Theorem	6
1.4. Bertrand Russell's Paradox	7
1.5. Entscheidungsproblem Problem	7
1.5.1. Describe the Entscheidungsproblem and how it relates to the Halting Problem	7
1.5.2. What proof by Absurbdum did Turing use in his proof of the Halting Problem?	8
1.5.3. What are the consequences of Turing's result?	9
1.5.4. How does this prove that the Entscheidungsproblem is undecidable? . .	9
2. Finite State Machines	9
2.1. The Main Components of a Finite State Machine	9
2.2. Describe the operation of a basic automatic door with pressure plates with a finite state machine	10
2.3. Describe the following finite state machine	10
2.4. Describe the following finite state machine	12
2.4.1. Design a FSM M_1 that only accepts strings containing even zeros . . .	13
2.4.2. Design a FSM M_2 that only accepts strings containing an odd number of 1s	13
2.4.3. An FSM M_3 whose language is the union of the languages of M_1 and M_2	14
2.5. Create a finite state machine that accepts the following string	14
2.5.1. Create a finite state machine that accepts the following string	14
2.5.2. Create a finite state machine that accepts the following string	14
2.5.3. Combine the two finite state machines to create a new finite state machine that accepts the following string	15
2.6. Prove that Regular languages are closed under union, concatenation, complement, and intersection	15
2.7. Main Advantages and Disadvantages of Finite State Machines	16
2.8. Pumping Lemma	17
2.8.1. Use the Pumping Lemma to show that $\{w w \text{ all strings end in } 11\}$ is regular	17
2.8.2. Use the Pumping Lemma to show that $\{0^n 1^n n \geq 0\}$ is not regular . . .	17
3. Turing Machines	18

3.1.	List all the differences between FSMs and modern day computers, and Turing Machines	18
3.2.	Define a Turing machine in at least 2 different ways	19
3.3.	Define a Universal Turing machine and give an example of one in modern computing	19
3.4.	Describe the following Turing machine	20
3.4.1.	Give a Formal description of the Turing machine	20
3.4.2.	Give the sequence of configurations for that the Turing Machine enters when started on the following inputs:	20
3.4.3.	Write down the top level description of a Turing machine that accepts the language $\{w\#w \mid w \in \{0,1\}^*\}$	21
3.4.4.	Give a formal description of the Turing machine that accepts the language $\{w\#w \mid w \in \{0,1\}^*\}$	21
3.5.	Design and formally describe a Turing machine that decides $B = \{0^{2^n} \mid n \geq 0\}$	22
3.5.1.	High Level description	22
3.5.2.	Formal description	22
3.5.3.	State Diagram	23
3.5.4.	A sample run of the machine on the input 0000	23
3.6.	Design a Turing Machine that accepts the language $C = \{a^i b^j c^k \mid i \times j = k \wedge i, j, k \geq 1\}$	24
3.6.1.	High Level description	24
3.6.2.	Formal description	24
3.7.	Review the formal definition of a Turing machine to answer the following questions, and explain your reasoning.	25
3.7.1.	Can a Turing machine ever write the blank symbol $\sqcup$ on its tape?	25
3.7.2.	Can the tape alphabet Γ be the same as the input alphabet Σ ?	25
3.7.3.	Can a Turing machine's head ever be in the same location in two successive steps?	25
3.7.4.	Can a Turing machine contain just a single state?	25
3.8.	Discuss two proposed hypercomputation models, and how they avoid the limitations of Turing machines.	25
3.9.	The Acceptance Problem	26
3.9.1.	Is the acceptance problem recognisable?	26
3.9.2.	Is the acceptance problem decidable?	27
3.10.	The Halting Problem Revisited	28
3.10.1.	The Formal Proof	28
3.11.	Validity and Significance of the Church-Turing thesis	29
3.12.	Can a Turing machine obtain Artificial Intelligence?	29
4.	Von Neumann Architecture	30
4.1.	Background	30
4.2.	Von Neumann Architecture	30
4.3.	Similarities and Differences between Turing Machines and Von Neumann Architecture	32

5. Lambda Calculus	34
5.1. Syntax	34
5.2. Identity Function	34
5.3. Handling Multiple Arguments	34
5.4. Beta Reductions	35
5.5. Church Encoding	35
5.5.1. Church Numerals	35
5.5.2. Successor Function	35
5.5.3. Church Arithmetic	36
5.6. Boolean Algebra	37
5.7. Combinators	38
5.8. The Y Combinator	39
6. Quantum Computing	40
6.1. Superposition (Why Qubits are Cool)	40
6.2. Tensor Product – The first Step to Qubits	41
6.3. Representing Multiple Qubits	41
6.4. Qubit Operations	42
6.4.1. Hadamard Gate	42
6.4.2. Pauli-X Gate (X):	43
6.4.3. CNOT Gate (CX):	43
6.5. Quantum Gate Matrices	43
6.6. Describe how qubits are constructed and how they compare to classical bits . .	44
6.7. What are the 4 main gates for a 2 qubit system?	45
6.8. Why is a Quantum Computer Better Than Conventional Computers in the context of the Deutsch Oracle Problem?	45
6.9. Practical Limitations	46

1. Mathematics for Computer Science

1.1. Proof $\sqrt{2}$ Is Irrational

We will utilise a **proof by contradiction** (also known as **reductio ad absurdum** – Reduce to absurdity) to show that $\sqrt{2}$ is an irrational number.

Rational number, formal definition:

$$\forall x \in \mathbb{R}, \exists a, b \in \mathbb{Z} \text{ where } x = \frac{a}{b} \quad (1.1)$$

This is denoted by $\mathbb{Q}$ which denotes all the rational numbers. An Irrational number is a number that cannot be expressed as a fraction of two integers, i.e., it is not in $\mathbb{Q}$. Or in other words:

$$\forall x \in \mathbb{R}, \quad x \notin \mathbb{Q} \quad \nexists a, b \text{ where } x = \frac{a}{b} \quad (1.2)$$

We wish to proceed by contradiction, so we assume that:

$\sqrt{2}$ is Rational i.e. $\sqrt{2} \in \mathbb{Q}$ where $a, b \in \mathbb{Z}$ and $\gcd(a, b) = 1$ (i.e., fraction is in lowest terms with no common factors). Now we can square both sides of the equation:

$$2 = \frac{a^2}{b^2} \rightarrow 2b^2 = a^2 \quad (1.3)$$

So a^2 is even, which implies a is even (as you can only get an even square if the original number is even), so we can write $a = 2k$ for some integer k . So:

$$2b^2 = (2k)^2 \quad (1.4)$$

$$2b^2 = 4k^2 \quad (1.5)$$

$$\Rightarrow b^2 = 2k^2 \quad (1.6)$$

So b^2 is even, which implies b is even. As a and b are both even, they have a common factor of 2 which contradicts our initial assumption (that the $\gcd(a, b) = 1$). Therefore, 2 cannot be a rational number.

1.2. Show that the cardinality of the Power Set $P(S)$ is larger than the cardinality of the set of natural numbers ($\mathbb{N}$)

We will utilise a **proof by contradiction** to show that the cardinality of the Power Set $P(S)$ is larger than the cardinality of the set of natural numbers ($\mathbb{N}$).

Power Set, formal definition:

$$P(S) = \{T \mid T \subseteq S\} \quad (1.7)$$

We can setup a table to compare the sizes of the two sets. We can tell that one set is larger than the other if we can show that the table is not square, or that there is a new row that cannot have appeared previously in the table after making $\mathbb{N}$ rows.

We will setup the table as follows:

	0	1	2	3	4	...
s_0	0	0	0	0	0	empty set
s_1	1	1	1	1	1	all numbers
s_2	1	0	1	0	1	odd numbers
s_3	0	1	0	1	0	even numbers
s_4	0	0	1	1	0	prime numbers
...						

Table 1.1: Table comparing the sizes of the Power Set $P(S)$ and $(\mathbb{N})$

We can define two sets as having the same size (or cardinality) if there exists some way to create a bijective mapping between them. In this case, we're looking at the natural numbers N and the powerset of the natural numbers $P(\mathbb{N})$. We assume that some bijective mapping between these two sets exists, so:

$$f : \mathbb{N} \rightarrow P(\mathbb{N}) \quad (1.8)$$

e.g. $0 \rightarrow S_0, 1 \rightarrow S_1, 2 \rightarrow S_2, 3 \rightarrow S_3, 4 \rightarrow S_4$, etc.

This maps every natural number to a subset of the natural numbers, and every subset has a number which maps to it. Consider the representation of these subsets (below is another (arbitrary example) of the representation like we did with 2c), where a 1 bit denotes that the natural number is included in the subset, and a 0 bit denotes that it is excluded (Note that we are assuming the rows of the table continue infinitely, and that every subset of the naturals exists in the table thanks to our bijective mapping).

We will now construct a new subset S that is not in the table, which will contradict our assumption that the mapping is bijective. S is defined by taking the diagonal of the table and flipping the bits, i.e. if the i th bit of S_i is 1, then the i th bit of S is 0, and vice versa.

As such, this new subset cannot appear in our table, as it differs from every row in the table by at least one bit. This means that there is no natural number that maps to this subset, which contradicts our assumption that the mapping is bijective. Therefore, we conclude that the cardinality of the Power Set $P(S)$ is larger than the cardinality of the set of natural numbers $(\mathbb{N})$.

Q.E.D.

1.3. Show Gödel's Incompleteness Theorem

1.3.1. Setup/Definitions

How does mathematics actually work? We have axioms, which are "a statement or proposition which is regarded as being established, accepted, or self-evidently true." E.g., $a + b = b + a$, things that are equal to the same thing are equal to each other, if $A=B$, $B=A$. Axioms are statements that are assumed to be true without requiring proof within the system. They serve as the starting point for deriving new results through logical deduction. In essence, axioms provide the basic building blocks upon which the entire structure of the

formal system is constructed. The goal was to have a set of axioms from which we can deduce all true statements. People also thought that we should be able to prove that mathematics is consistent (the mathematics won't give rise to contradictions).

A **formal** system is **complete** if all the theorems can be either **proved** or **disproved**; A system is **complete** if the axioms **govern all** of the possible theorems. It is **consistent** if no two theorems contradict each other (if there is no statement such that both the statement and its negation are provable from the axioms, and inconsistent otherwise).

Gödel numbering is a method of encoding mathematical expressions, statements, and even proofs as unique natural numbers. Each symbol (like "+", "=", "x", etc.) is assigned a unique number, and sequences of symbols (i.e., entire formulas or proofs) are encoded using a method like prime factorization. This encoding allows us to treat statements about mathematics as mathematical objects themselves. That is, we can now talk within mathematics about statements, proofs, and even the process of proving things. It allows meta-mathematical reasoning within a formal system. We can now ask: "Can this statement be proved in the system?" as a mathematical question. It provides insight into the limits of formal systems, which is key for Gödel's incompleteness theorems.

1.3.2. Proof of Gödel's Incompleteness Theorem

Let's prove Gödel's first incompleteness theorem:

Gödel's incompleteness theorem I: There are statements of consistent formal system which can neither be proved nor disproved.

So now, using Gödel numbering, we make a statement g saying "there is no proof for statement g – this statement cannot be proved from the axioms".

If g is true, then the mathematical system has true statements (i.e. the statement g) that have no proof. This means that the system is incomplete as a complete system is defined such that it must be able to prove or disprove all its statements. No contradiction!

If g is false, this means that the statement **IS** provable with the axioms, but a provable statement must be true (if we want a consistent system). So we started by saying it is False, and ended up saying it is True $\Rightarrow \Leftarrow$ Contradiction. i.e., there is proof for g that there is no proof for g . We are assuming that mathematics is consistent so we can't have contradictions. Therefore, g can't be False, and therefore must be True. (Contrast with Liar Paradox: "this sentence is false" cannot be shown to be either true nor false using logic. This is a linguistic paradox, which doesn't have a truth value.)

In Conclusion, the statement g **must** be true, and therefore there are statements in mathematics that cannot be proved or disproved. This is Gödel's first incompleteness theorem.

1.3.3. Proof of Gödel's Second Incompleteness Theorem

Gödel's incompleteness theorem II: Any consistent formal system of math cannot prove its own consistency.

Assume formal system S is consistent. From the first theorem, we know that there is a statement g in S that cannot be proved or disproved. If S could prove its own consistency (no two theorems contradict each other), then it could prove g (which is a statement in S). If a system can prove its own consistency, then there would exist a Gödel number for the statement “This system is consistent”. This would then imply that the system can prove g , which contradicts the first theorem. Therefore, S cannot prove its own consistency.

1.4. Bertrand Russell's Paradox

Russell's Paradox is a famous paradox in set theory that arises when we consider the set of all sets that do not contain themselves. It was discovered by Bertrand Russell in 1901 and highlights a fundamental problem in naive set theory. The paradox can be summarised as follows:

Let R be the set of all sets that do not contain themselves. Formally, we can define R as:

$$R = \{S \mid S \notin S\} \quad (1.9)$$

Now, we can ask the question: Does R contain itself? If R contains itself, then by the definition of R , it must not contain itself (since it only contains sets that do not contain themselves). This leads to a contradiction. Likewise, if R does not contain itself, then it must contain itself (since it is a set that does not contain itself). This also leads to a contradiction.

Therefore, we conclude that the set R cannot exist without leading to a contradiction. This constitutes a paradox.

What's the point? Russell's paradox is a standard way to show naïve set theory is flawed. Naïve set theory uses the comprehension principle. This states that ‘given any property there exists a set containing all objects that have that property’. But we know that not every set is valid (because paradox). So we have to be careful with how we define and construct a set.

1.5. Entscheidungsproblem Problem

1.5.1. Describe the Entscheidungsproblem and how it relates to the Halting Problem

Entscheidungsproblem = “Decision Problem” in German – Can we determine using an algorithm if any given **statement** is provable or non-provable?

This is very similar to the Halting Problem – Can we determine if any given **program** given any input will halt or not? (Decidable or undecidable)

- These are the same problem. A program is a sequence of 1s and 0s. Any sequence of operation on numbers is a mathematical statement. Decision problem is a more general question about the existence of a decision algorithm for mathematical statements, and halting problem is more specific version of the decision problem related to the behaviour of computer programs.

- Example: Take a computer program P and input I. You can construct a corresponding mathematical statement S that says, 'Program P halts on input I'. The Decision problem is, "is this mathematical statement (S) true or false?"

1.5.2. What proof by Absurbdum did Turing use in his proof of the Halting Problem?

Turing used a proof by contradiction (reductio ad absurdum) to show that the Halting Problem is undecidable. Lets see how this works:

Let's assume that we have some machine that solves the halting problem. So Turing started by making a machine (a universal Turing Machine – Called machine H) that takes program P and input I and determines if P will halt (i.e., it will give us an answer) on input I or not.

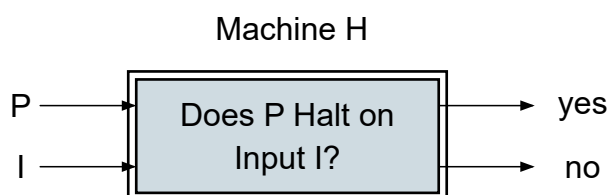


Figure 1.1: Turing's Machine H

So H takes input P and I and says "Yes" it will halt, or "No" it will not halt. So now, we want to make a new machine H+ that, if H gives a "Yes" then H+ will loop forever and if H gives a "No" then H+ will halt. This essentially means that H+ does the opposite of what H says.

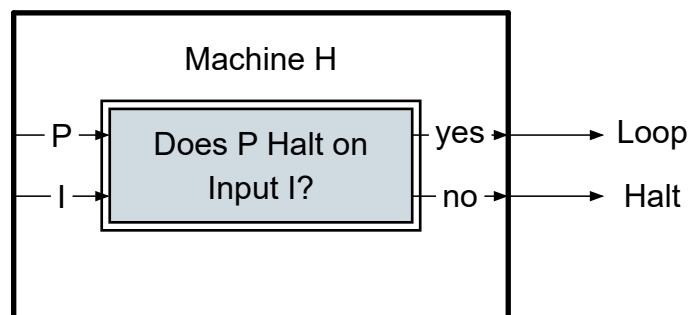


Figure 1.2: Turing's Machine H+

What happens if we feed H+ into itself? So we give H+ to itself, H+ feed it into the H inside itself, so H asks, "Does H+ halt on input H+?".

What happens if we feed H+ into itself? So we give H+ to itself, H+ feed it into the H inside itself, so H asks, "Does H+ halt on input H+?".

If H says that H+ does halt, we get a Yes output from H, but then H+ loops (doesn't halt). If H says that H+ doesn't halt, we get a No output from H, but then H+ halts.

So if H+ does halt, then it doesn't halt, and if it doesn't halt, then it halts. Either way, we have a contradiction.

So, we started off by assuming we could solve the problem, we found a contradiction, so we must reject our assumption. Therefore, there is no algorithm that solves the halting problem.

1.5.3. What are the consequences of Turing's result?

The significance is that, your computer might be running something that will never halt, but your computer cannot say for certain that it won't halt. So you need to use some sort of time-out mechanism (e.g., after too many loops). There are many problems that computers can't solve.

1.5.4. How does this prove that the Entscheidungsproblem is undecidable?

The Halting Problem is a decision problem where the input consists of a program (or Turing Machine) and an input value for that program. The question it poses is: Will the program halt when given that input, or will it run forever? Alan Turing proved that the Halting Problem is undecidable, meaning no algorithm can universally determine the halting behaviour for all possible inputs. Now, suppose you make another machine that can solve a different problem: deciding whether any first-order logic statement is true or false (this is called the decision problem for first-order logic). If such a machine existed, we could cleverly use it to also solve the Halting Problem—which we already know is impossible. This leads us to an unavoidable conclusion: the decision problem for first-order logic must also be undecidable, because if it were solvable, the Halting Problem would be too, which contradicts Turing's proof. In other words, proving the Halting Problem is undecidable lets us transfer that insolvability to other problems by reducing them to it.

2. Finite State Machines

2.1. The Main Components of a Finite State Machine

A Finite State Machine (FSM) is a computational model used to design algorithms and systems that can be in one of a finite number of states at any given time. The main components of an FSM are:

- Character Input: The input symbols that the FSM can read. They are single symbols from a specified alphabet denoted Σ or A
 - The set of all possible strings in the alphabet is denoted A^* or Σ^* . This also includes the empty string.
- States form a set Q or S
 - Start state: The state where the FSM begins its operation, denoted $q_0 \in Q$ or $s_0 \in S$
 - Accept state: A state that indicates the FSM has successfully processed an input string, denoted $F \in Q$ or $s_{\text{yes}} \in S$
- Transition function: A function that defines how the FSM moves from one state to another based on the input symbols. It is denoted as $\delta : Q \times \Sigma \rightarrow Q$ or $\delta : S \times A \rightarrow S$

The language accepted by an FSM represents a decision whether some input satisfies some boolean property. Note that by definition, the start state and the accept state cannot be the same state.

The state transition function is better represented as a table or diagram than a function, as it is easier to visualise.

2.2. Describe the operation of a basic automatic door with pressure plates with a finite state machine

- Door states: open, closed – at minimum
- Behaviour:
 - If the door is closed and a person steps on the pressure plate, the door opens.
 - If the door is open and a person steps on the pressure plate, the door stays open.
 - If the door is open and no one is on the pressure plate, the door closes.
 - If the door is closed and no one is on the pressure plate, the door stays closed.
- Alphabet: {on pressure plate, not on pressure plate}
- States: {open, closed}
- Note: start and accept states don't make sense for this example, as the door can be in either state at any time.
- Transition diagram:

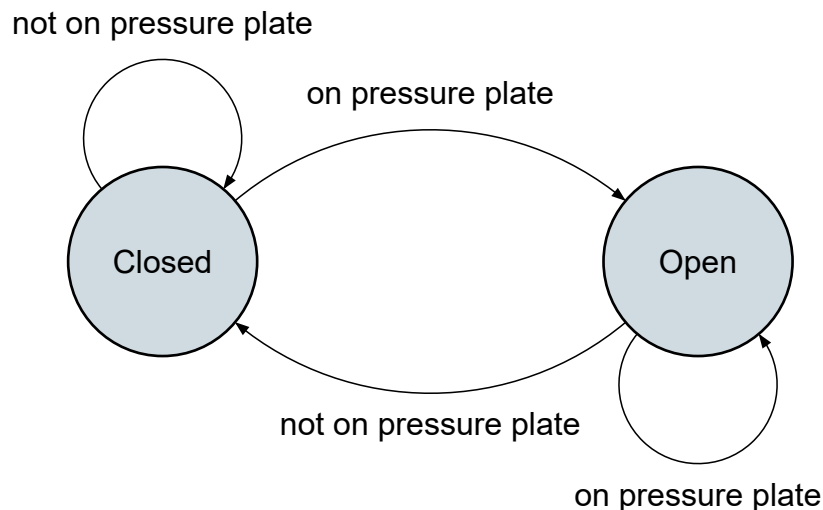


Figure 2.1: Finite State Machine for a basic automatic door with pressure plates

2.3. Describe the following finite state machine

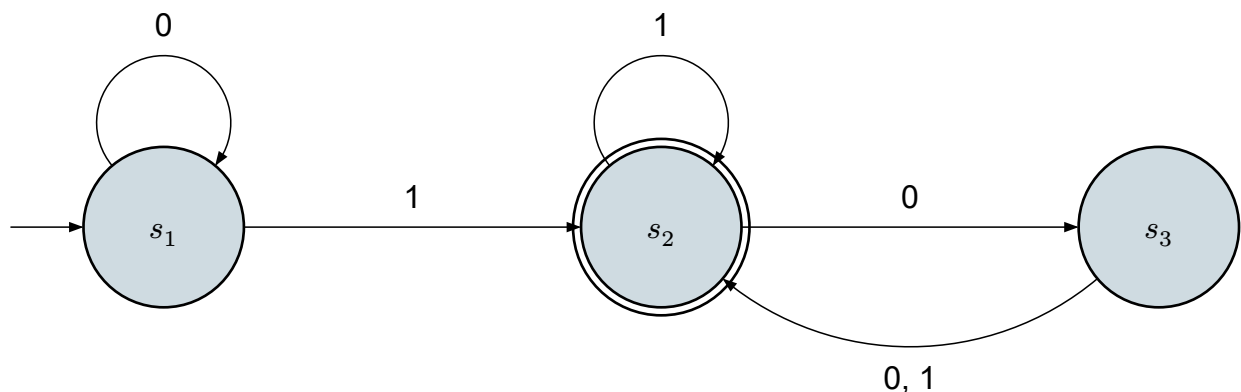


Figure 2.2: Finite State Machine with states s_1 , s_2 , and s_3

1. List the set of all possible states:

- $S = \{s_1, s_2, s_3\}$
- s_1 – Start state
- s_2 – Accept state

2. Write the transition table:

Current State	Next State	
	0	1
s_1	s_1	s_2
s_2	s_3	s_2
s_3	s_2	s_3

Table 2.1: Transition table for the finite state machine with states s_1 , s_2 , and s_3

3. Show that it accepts the strings $w = 10100$ and $u = 0001$

w : begin in s_1 , read 1 $\rightarrow s_2$, read 0 $\rightarrow s_3$, read 1 $\rightarrow s_2$, read 0 $\rightarrow s_3$, read 0 $\rightarrow s_2$ (accept state). So it accepts the string $w = 10100$. u : begin in s_1 , read 0 $\rightarrow s_1$, read 0 $\rightarrow s_1$, read 0 $\rightarrow s_1$, read 1 $\rightarrow s_2$ (accept state). So it accepts the string $u = 0001$.

4. What sort of strings does it accept?

- It accepts strings that have an even number of 0's after the last 1.
- In regular expression notation, this can be expressed as:
 - $0^*1((00)|(01)|(1))^*$
 - or more simply: $.^*1(00)^*$
- Quick breakdown of regular languages:
 - $.^*1$ is greedy, so it will match as many instances as it can until it reaches the **final** instance of 1.
 - $.^*?1$ is not greedy, it will match all instances until it reaches the **first** instance of 1.

2.4. Describe the following finite state machine

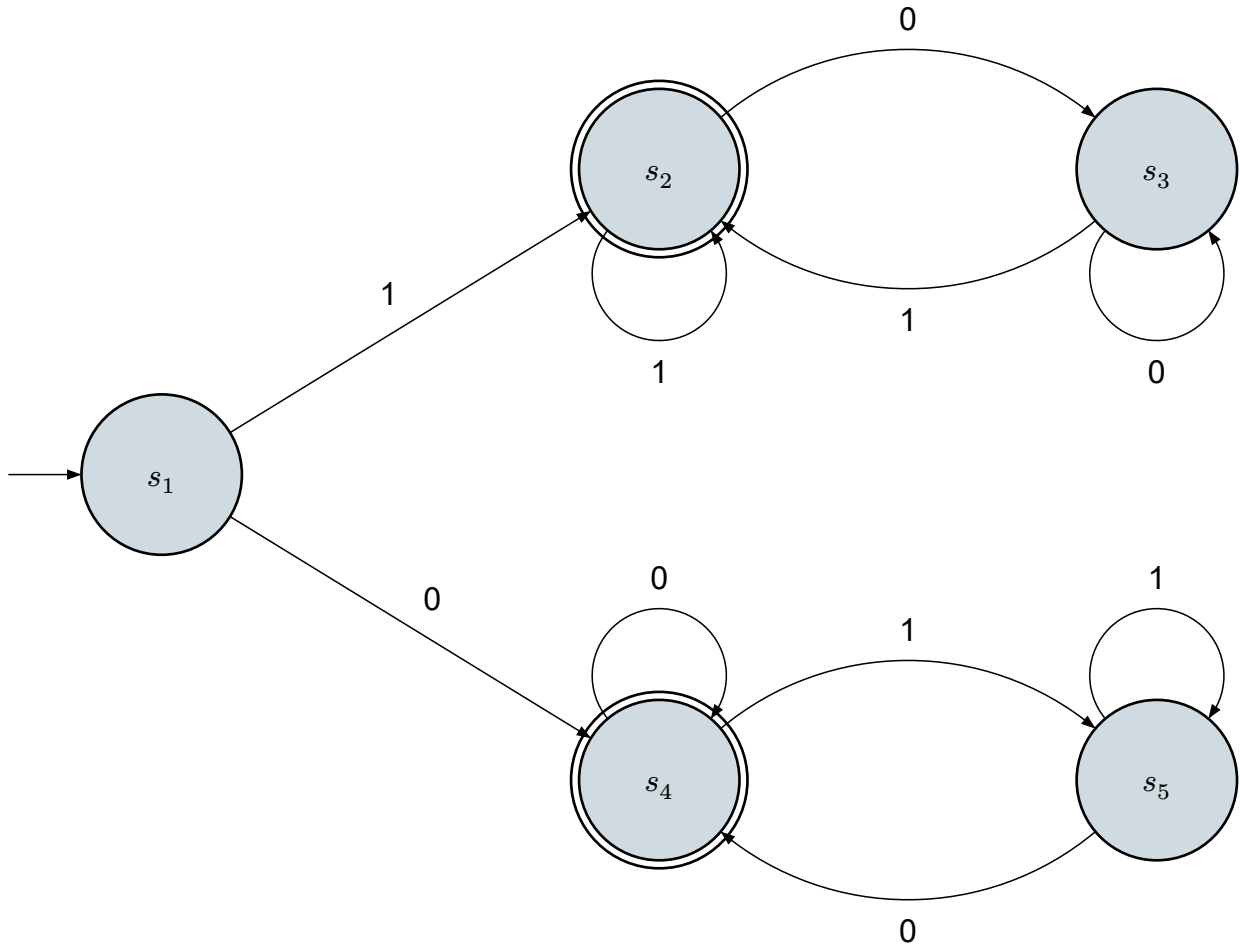


Figure 2.3: Finite State Machine with states s_1, s_2, s_3, s_4 , and s_5

1. List the set of all possible states:

- $S = \{s_1, s_2, s_3, s_4, s_5\}$
- s_1 – Start state
- s_2, s_4 – Accept state

2. Write the transition table:

Current State	Next State	
	0	1
s_1	s_4	s_2
s_2	s_3	s_2
s_3	s_3	s_2
s_4	s_4	s_5
s_5	s_4	s_5

Table 2.2: Transition table for the finite state machine with states s_1, s_2, s_3, s_4 , and s_5

3. Show that it accepts the strings $w = 101$ and $u = 00$

w:

1. $\delta(s_1, 1) = s_2$
2. $\delta(s_2, 0) = s_3$

3. $\delta(s_3, 1) = s_2$

u:

1. $\delta(s_1, 0) = s_4$

2. $\delta(s_1, 0) = s_4$

4. What type of strings does it accept? It will accept strings that start and end with the same character. In regular expression notation, this can be expressed as: $1|0|(1.*1)|(0.*0)$

2.4.1. Design a FSM M_1 that only accepts strings containing even zeros

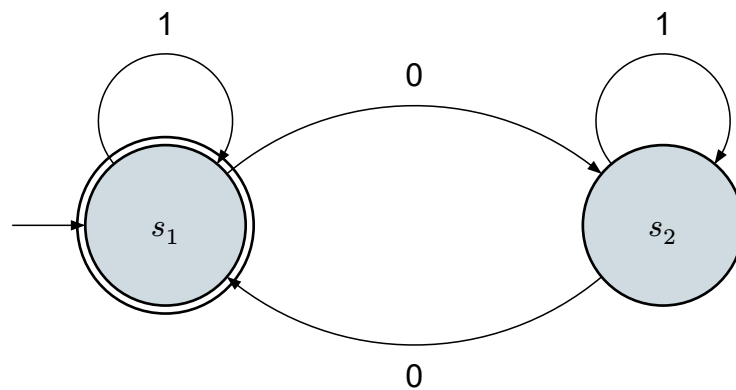


Figure 2.4: Finite State Machine M_1 that only accepts strings containing even zeros

Regex notation: $((00)|(1))^*$

2.4.2. Design a FSM M_2 that only accepts strings containing an odd number of 1s

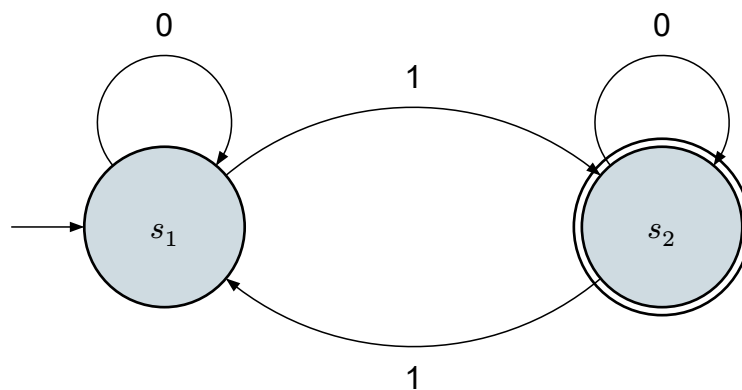


Figure 2.5: Finite State Machine M_2 that only accepts strings containing an odd number of 1s

Regex notation: $(0^*(10^*10^*)^*10^*)$

2.4.3. An FSM M_3 whose language is the union of the languages of M_1 and M_2

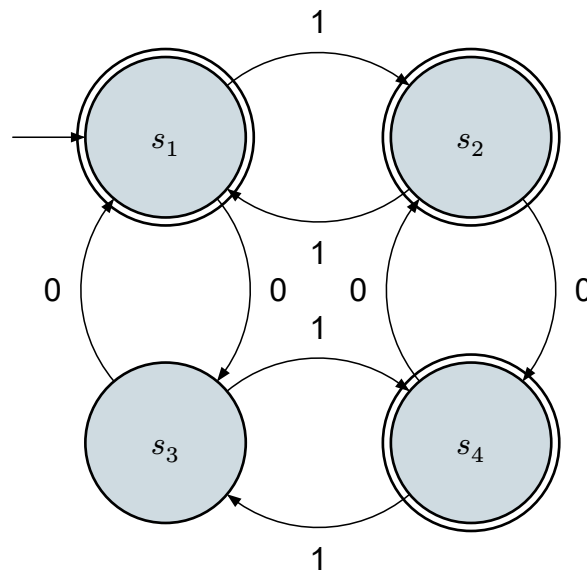


Figure 2.6: Finite State Machine $M_3 = M_1 \cup M_2$

Regex notation: $((00)|(1))^*|(0^*(10^*10^*)^*10^*)$

2.5. Create a finite state machine that accepts the following string

$\{w \mid x \in w \text{ has exactly two 0s and at least two 1s}\}$

We first break the problem down into smaller parts. First we create a FSM which accepts strings with exactly two 0s, and then we create a FSM which accepts strings with at least two 1s. Finally, we combine the two FSMs to create a new FSM which accepts strings that satisfy both conditions.

2.5.1. Create a finite state machine that accepts the following string

$\{w \mid x \in w \text{ has exactly two 0s}\}$

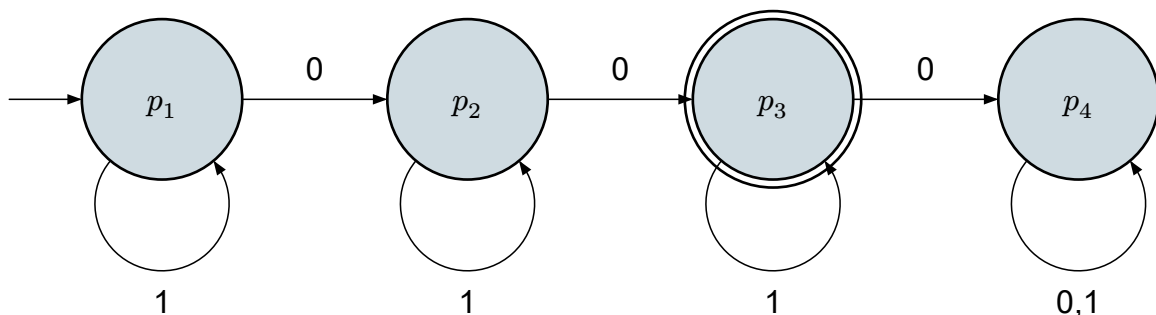


Figure 2.7: Finite State Machine that accepts strings with exactly two 0s

2.5.2. Create a finite state machine that accepts the following string

$\{w \mid x \in w \text{ has at least two 1s}\}$

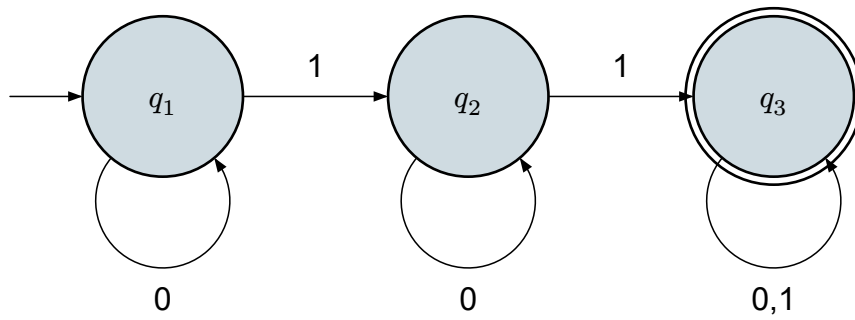


Figure 2.8: Finite State Machine that accepts strings with at least two 1s

2.5.3. Combine the two finite state machines to create a new finite state machine that accepts the following string

$\{w \mid x \in w \text{ has exactly two 0s and at least two 1s}\}$

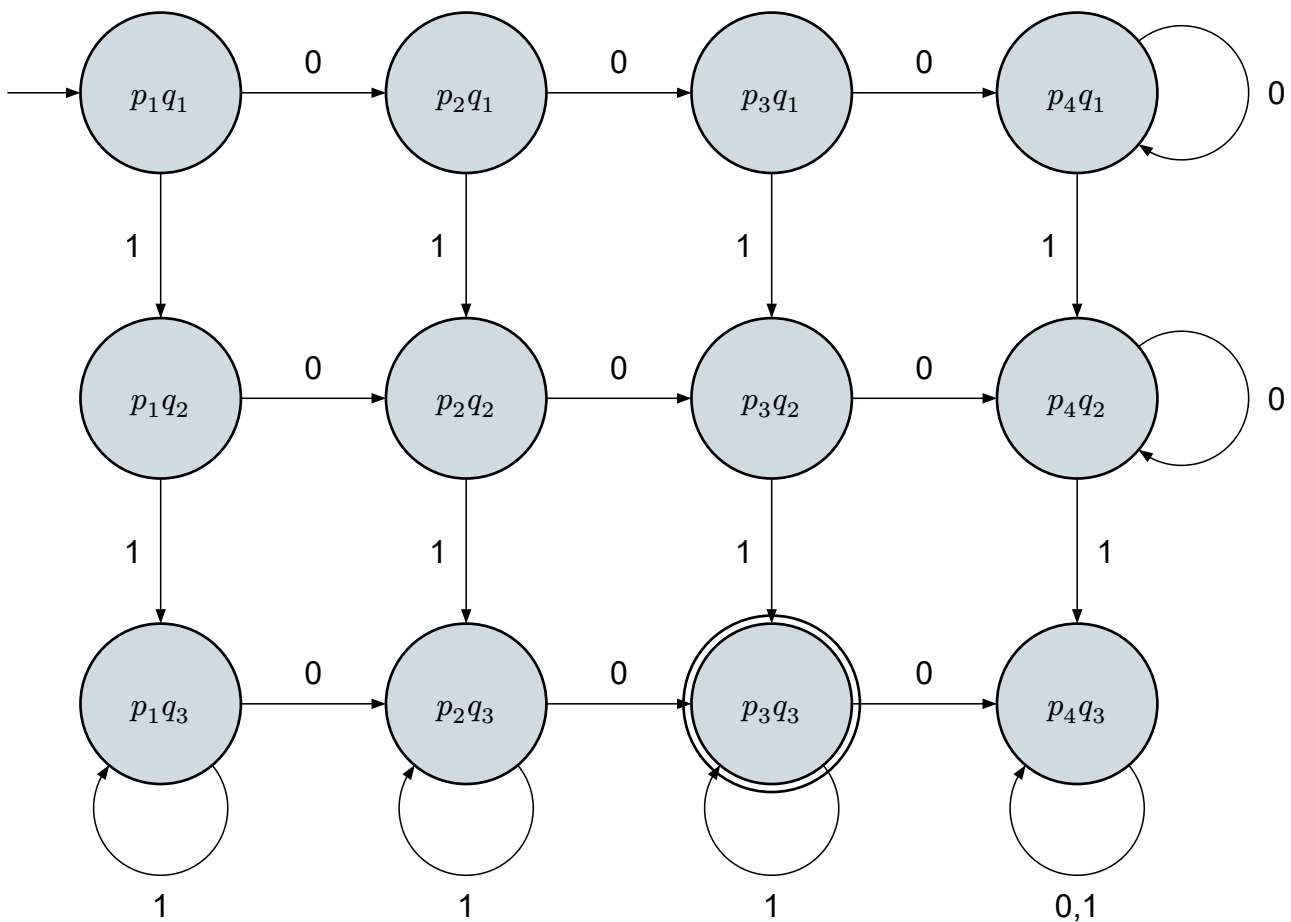


Figure 2.9: Finite State Machine that accepts strings with exactly two 0s and at least two 1s

Regex expression: $1^*(0011|0101|0110|1001|1010|1100)1^*$

2.6. Prove that Regular languages are closed under union, concatenation, complement, and intersection

$M_1 \cup M_2$

Proof by construction, we construct M from $M_1 \cup M_2$ where M recognises L if M_1 accepts L , and M_2 accepts L .

1. Let M_1 recognise L_1 where $\{M_1 = Q_1, \sum_1, \delta_1, q_1, F_1\}$
2. Let M_2 recognise L_2 where $\{M_2 = Q_2, \sum_2, \delta_2, q_2, F_2\}$

Construct M to recognise $L_1 \cup L_2$ where $M = \{Q, \sum, \delta, q_1, F\}$ where:

- Q is the cartesian product of sets $Q_1, Q_2 : Q_1 \times Q_2$
- $\sum$ is the alphabet and is the same as $\sum_1 \times \sum_2$
- δ : where $\delta((r_1, r_2), a) = (\delta(r_1, a), \delta(r_2, a))$ "for" $r_1 \in Q_1, r_2 \in Q_2, a \in \sum$
- q_1 : the pair (q_1, q_2)
- F : where $(F_1 \times Q_2) \cup (F_2 \times Q_1)$

Thus we have that if L_1 & L_2 are regular then $L_1 \cup L_2$ is also regular $\square$

$M_1 \cap M_2$

Proof by construction, we construct M from $M_1 \cap M_2$ where M recognises L if M_1 accepts L , and M_2 accepts L .

1. Let M_1 recognise L_1 where $\{M_1 = S_1, \sum_1, \delta_1, q_1, F_1\}$
2. Let M_2 recognise L_2 where $\{M_2 = S_2, \sum_2, \delta_2, q_2, F_2\}$

Construct M to recognise $L_1 \cap L_2$ where $M = \{Q, \sum, \delta, q_1, F\}$ where:

- Q is the cartesian product of sets $Q_1, Q_2 : Q_1 \times Q_2$
- $\sum$ is the alphabet and is the same as $\sum_1 \times \sum_2$
- δ : where $\delta((r_1, r_2), a) = (\delta(r_1, a), \delta(r_2, a))$ "for" $r_1 \in Q_1, r_2 \in Q_2, a \in \sum$
- q_1 : the pair (q_1, q_2)
- F : where $(F_1 \times F_2)$

Thus we have that if L_1 & L_2 are regular then $L_1 \cap L_2$ is also regular $\square$

M_1^c

Proof by construction, we construct M from M_1^c where M recognises L if M_1 accepts L .

1. Let M_1 recognise L_1 where $\{M_1 = S_1, \sum_1, \delta_1, q_1, F_1\}$

Construct M to recognise L_1^c where $M = \{Q, \sum, \delta, q_1, F\}$ where:

- Q is the set of all states in M_1
- $\sum$ is the alphabet and is the same as $\sum_1$
- δ : where $\delta(r, a) = \delta_1(r, a)$ "for" $r \in Q_1, a \in \sum$
- q_1 : the start state q_1
- F : where $F = Q_1 \setminus F_1$ (the set of all states in M_1 minus the accept states or in other words, the complement of the accept states)

Thus we have that if L_1 is regular then L_1^c is also regular $\square$

2.7. Main Advantages and Disadvantages of Finite State Machines

Advantages

- Often small transition table fits in < 10kB.
Usually very little ram to store current state
- simple execution code:

Disadvantages

- Trouble handling complex operations requiring multiple inputs at a time e.g. game FSM vs GOAL


```
for i in (input_string):
    current_state =
    transition_table[current_state][i]
```

- Low complexity means fast resolution on modern hardware
- Domain specific language: regex

- Difficulty in representing arbitrary set of accept states, e.g. English language
- Only modular arithmetic for maths
- only knows current state $\rightarrow$ finite set of equivalence classes
- some problems are impossible, e.g. bracket matching, palindromes, etc.

2.8. Pumping Lemma

2.8.1. Use the Pumping Lemma to show that $\{w|w \text{ all strings end in } 11\}$ is regular

To show that the language $\{w|w \text{ all strings end in } 11\}$ is regular using the Pumping Lemma, we need to demonstrate that it satisfies the conditions of the lemma.

Set $L = \{w|w \text{ all strings end in } 11\}$. let $s \in L$ be a string of length $|s| \geq p$ where p is the pumping length. The Pumping Lemma states that there exists a decomposition of s into three parts, $s = xyz$, such that:

1. $|y| > 0$ (the length of y is greater than 0)
2. $|xy| \leq p$ (the length of the concatenation of x and y is less than or equal to p)
3. For all $i \geq 0$, the string xy^iz is in L (the string remains in the language when y is repeated any number of times, including zero)

As an example, consider a string $s = 0011$ where the 3 conditions are satisfied ($|s| = 4 \geq p$):

- Let $x = 0$, $y = 0$, and $z = 11$.
- $|y| = 1 > 0$ (condition 1)
- $|xy| = |00| = 2 \leq p$ (condition 2)
- For all $i \geq 0$, the string xy^iz is in L :
 - For $i = 0$: $xy^0z = 011$ (ends in 11, so in L)
 - For $i = 1$: $xy^1z = 0011$ (ends in 11, so in L)
 - For $i = 2$: $xy^2z = 00011$ (ends in 11, so in L)

Thus, the language $\{w|w \text{ all strings end in } 11\}$ satisfies the conditions of the Pumping Lemma. However this does not prove that the language is regular, it only shows that it satisfies the conditions of the Pumping Lemma. The pumping lemma can only be used to show that a language is not regular, not that it is regular.

2.8.2. Use the Pumping Lemma to show that $\{0^n 1^n | n \geq 0\}$ is not regular

Assume that this language (denoted B) is regular and P be some pumping length. Let $s = 0^p 1^p$. As s is a string accepted by the language and has length greater than p , we can split this into x = before the pump, y = the substring to be pumped and z = the accepted state. Consider three cases:

1. If y consists of only 0's, then pumping y will result in more 0's than 1's, which is not in B .
2. If y consists of only 1's, then pumping y will result in more 1's than 0's, which is not in B .

3. If y consists of both 0's and 1's, then pumping y may disrupt the balance between 0's and 1's, which is not in B .

As there is a case where pumping y results in a string that is not in B , we have a contradiction. Therefore, the language $\{0^{1^n} | n \geq 0\}$ is not regular.

3. Turing Machines

3.1. List all the differences between FSMs and modern day computers, and Turing Machines

FSM	TM	PC
Hypothetical	Hypothetical	Physical
States and transitions only (No memory)	Infinite memory (tape)	Finite memory
transition rules $\delta : \{S\} \times \{A\} \rightarrow \{S\}$ (Read $\rightarrow$ Head: L/R)	$\delta : \{S\} \times \{A\}_{\text{Tape}} \rightarrow \{S\} \times \{A\}_{\text{Tape}} \times \{L, R\}$ (Read $\rightarrow$ Write $\rightarrow$ Head: L/R)	Read/write
Limited run time	Unlimited run time	Limited run time?
Describes the class of regular languages	Accepts a much larger class of languages	Any n , as long as it does not run out of memory?

Table 3.1: Comparison of FSM, Turing Machine, and PC

- **FSM (Finite State Machine) – Hypothetical:** FSMs are theoretical models used in computer science and automata theory to represent computation with a limited number of states. They are not physically implemented in isolation but serve as conceptual tools for understanding simpler computational systems.
- **TM (Turing Machine) – Hypothetical:**
Turing Machines are abstract mathematical models introduced by Alan Turing. They can simulate any algorithm's logic, serving as the foundation for understanding computation. However, they are not built as real machines in their pure form; they exist as theoretical constructs.
- **PC (Personal Computer) – Physical:**
A PC is a real, tangible machine that embodies the concepts of both FSMs and TMs. While it has finite memory (unlike a true Turing Machine), it can simulate a Turing Machine to a large extent and is physically implemented with hardware.
- **FSM (Finite State Machine) – Limited run time:**
FSMs process input step by step and halt once the input ends. Their computation is linear and bound by the input size, meaning they can't go on indefinitely.
- **TM (Turing Machine) – Unlimited run time:**

Turing Machines can, in theory, run forever especially for computations with no halting condition. They are not bound by input size and have infinite tape, so their runtime is considered unbounded in theory.

- PC (Personal Computer) – Limited run time?:

The question mark suggests some ambiguity. In practice:

- PCs are physical and constrained by power, memory, and processing limits, so they can't run forever.
- However, they can simulate long-running processes or approximate Turing Machines for practical purposes.

3.2. Define a Turing machine in at least 2 different ways

Formal definition: A Turing machine is defined as a 7-tuple $M =$

$(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ where:

- Q is a finite set of states
- Σ is the input alphabet
- Γ is the tape alphabet
- δ is the transition function
- q_0 is the start state
- q_{accept} is the accept state
- q_{reject} is the reject state

Informal definition: A Turing machine is a theoretical computational model that consists of an infinite tape divided into cells, a read/write head that can move left or right, and a finite set of states. It processes input by reading symbols from the tape, writing symbols back to the tape, and transitioning between states based on a set of rules defined by its transition function.

3.3. Define a Universal Turing machine and give an example of one in modern computing

UTM's are turing machines that can simulate any other Turing machine. They take as input a description of another Turing machine and its input, and they simulate the behavior of that Turing machine on that input. As such, the other turing machine must be encoded in the tape alphabet of the UTM.

- A modern day example of a UTM is an operating system (OS) that can run multiple applications, each of which can be thought of as a Turing machine. The OS manages the resources and provides an environment for these applications to run, effectively simulating the behavior of each application as if it were a Turing machine.
- A more specific example is an emulator which replicates the functionality of a different computer system, allowing software designed for that system to run on a different architecture.
- And of course the CPU, which can execute any program that is written in a language that can be compiled to machine code, effectively simulating a Turing machine.

3.4. Describe the following Turing machine

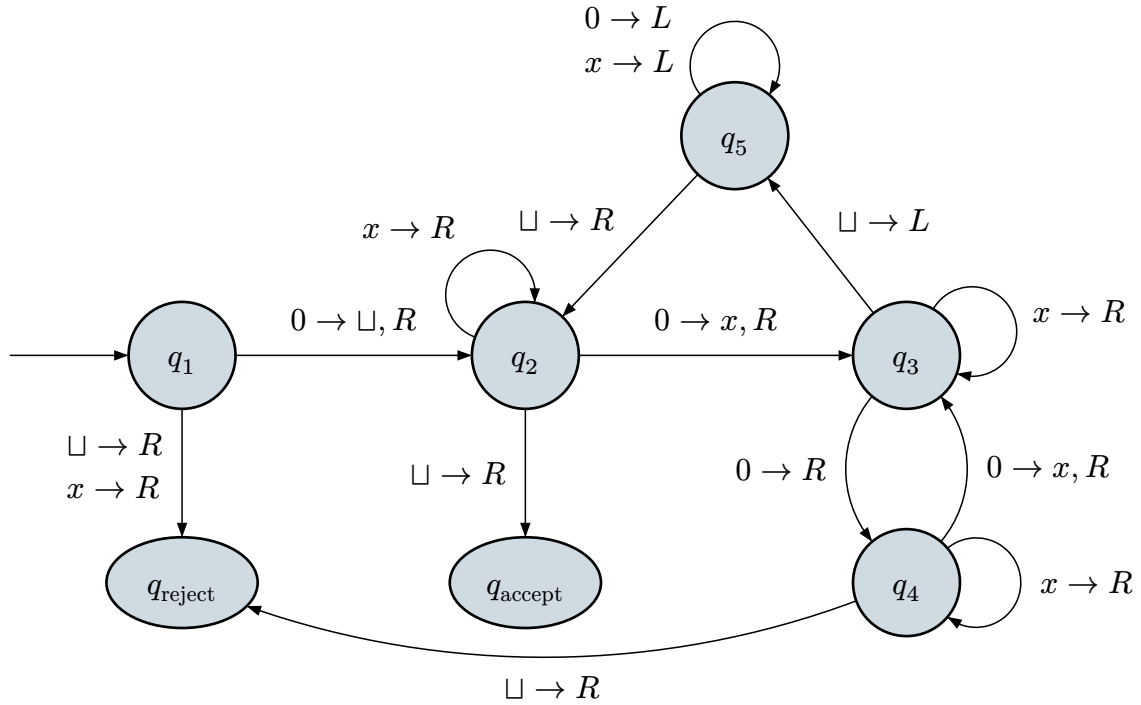


Figure 3.1: Turing Machine with states $q_1, q_2, q_3, q_4, q_5, q_{\text{accept}},$ and q_{reject}

3.4.1. Give a Formal description of the Turing machine

Let $M = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$ be a Turing machine where:

- $Q = \{q_1, q_2, q_3, q_4, q_5, q_{\text{accept}}, q_{\text{reject}}\}$ is the set of states
- $\Sigma = \{0\}$ is the input alphabet
- $\Gamma = \{0, x, \sqcup\}$ is the tape alphabet
- start state: q_1
- Accept state: q_{accept}
- Reject state: q_{reject}

3.4.2. Give the sequence of configurations for that the Turing Machine enters when started on the following inputs:

recall that the notation for a turing configuration is that the state is written as part of the tape, and the head position is denoted by the state name. The head is always reading the character immediately to the right of the state name.

- | | | | |
|--------------------------------------|--|--|-----------------------------------|
| a. 1. 0 | 2. 00 | 3. 000 | 4. 000000 |
| 1. $q_1 0$ | 1. $q_1 00$ | 1. $q_1 000$ | 1. $q_1 000000$ |
| 2. $\sqcup q_2 \sqcup$ | 2. $\sqcup q_2 0$ | 2. $\sqcup q_2 00$ | 2. $\sqcup q_2 000000$ |
| 3. $\sqcup \sqcup q_{\text{accept}}$ | 3. $\sqcup x q_3 \sqcup$ | 3. $\sqcup x q_3 0$ | 3. $\sqcup x q_3 0000$ |
| | 4. $\sqcup q_5 x \sqcup$ | 4. $\sqcup x 0 q_4 \sqcup$ | 4. $\sqcup x 0 q_4 000$ |
| | 5. $q_5 \sqcup x \sqcup$ | 5. $\sqcup x 0 \sqcup q_{\text{reject}}$ | 5. $\sqcup x 0 x q_3 00$ |
| | 6. $\sqcup q_2 x \sqcup$ | | 6. $\sqcup x 0 x 0 q_4 0$ |
| | 7. $\sqcup x q_2 \sqcup$ | | 7. $\sqcup x 0 x 0 x q_3 \sqcup$ |
| | 8. $\sqcup x \sqcup q_{\text{accept}}$ | | 8. $\sqcup x 0 x 0 q_5 x \sqcup$ |
| | | | 9. $\sqcup x 0 x q_5 0 x \sqcup$ |
| | | | 10. $\sqcup x 0 q_5 x 0 x \sqcup$ |

11. $\sqcup xq_5 0x0x \sqcup$
12. $\sqcup q_5 x0x0x \sqcup$
13. $q_5 \sqcup x0x0x \sqcup$
14. $\sqcup q_2 x0x0x \sqcup$
15. $\sqcup xq_2 0x0x \sqcup$
16. $\sqcup xxq_3 x0x \sqcup$
17. $\sqcup xxxq_3 0x \sqcup$
18. $\sqcup xxx0q_4 x \sqcup$
19. $\sqcup xxx0xq_4 \sqcup$
20. $\sqcup xxx0x \sqcup q_{\text{reject}}$

3.4.3. Write down the top level description of a Turing machine that accepts the language $\{w\#w \mid w \in \{0,1\}^*\}$

The main algorithm:

1. At the start the tape is at the leftmost cell
2. read the first symbol and “remember” if it was a 1 or a 0 using states, and cross it off by replacing it with an “x”
3. move to the right until you find a #, then keep on moving until you find a symbol that is not an “x”.
4. If you find a symbol that is not an “x”, then check if it is the same as the first symbol you read. If it is, then cross it off by replacing it with an “x” and move back to the leftmost cell and repeat the process.
5. If you find a symbol that is not the same as the first symbol you read, then reject the input.

If: the number of #'s is not equal to 1, there are different symbols, the number of symbols before the # is not equal to the number of symbols after the # then reject the input.

3.4.4. Give a formal description of the Turing machine that accepts the language $\{w\#w \mid w \in \{0,1\}^*\}$

Let $M = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$ be a Turing machine where:

- $Q = \{q_1, q_2, q_3, q_4, q_5, q_{\text{accept}}, q_{\text{reject}}\}$ is the set of states
- $\Sigma = \{0, 1, \#\}$ is the input alphabet
- $\Gamma = \{0, 1, \#, x\}$ is the tape alphabet
- start state: q_1
- Accept state: q_{accept}
- Reject state: q_{reject}

The transition function δ is defined by the following state diagram:

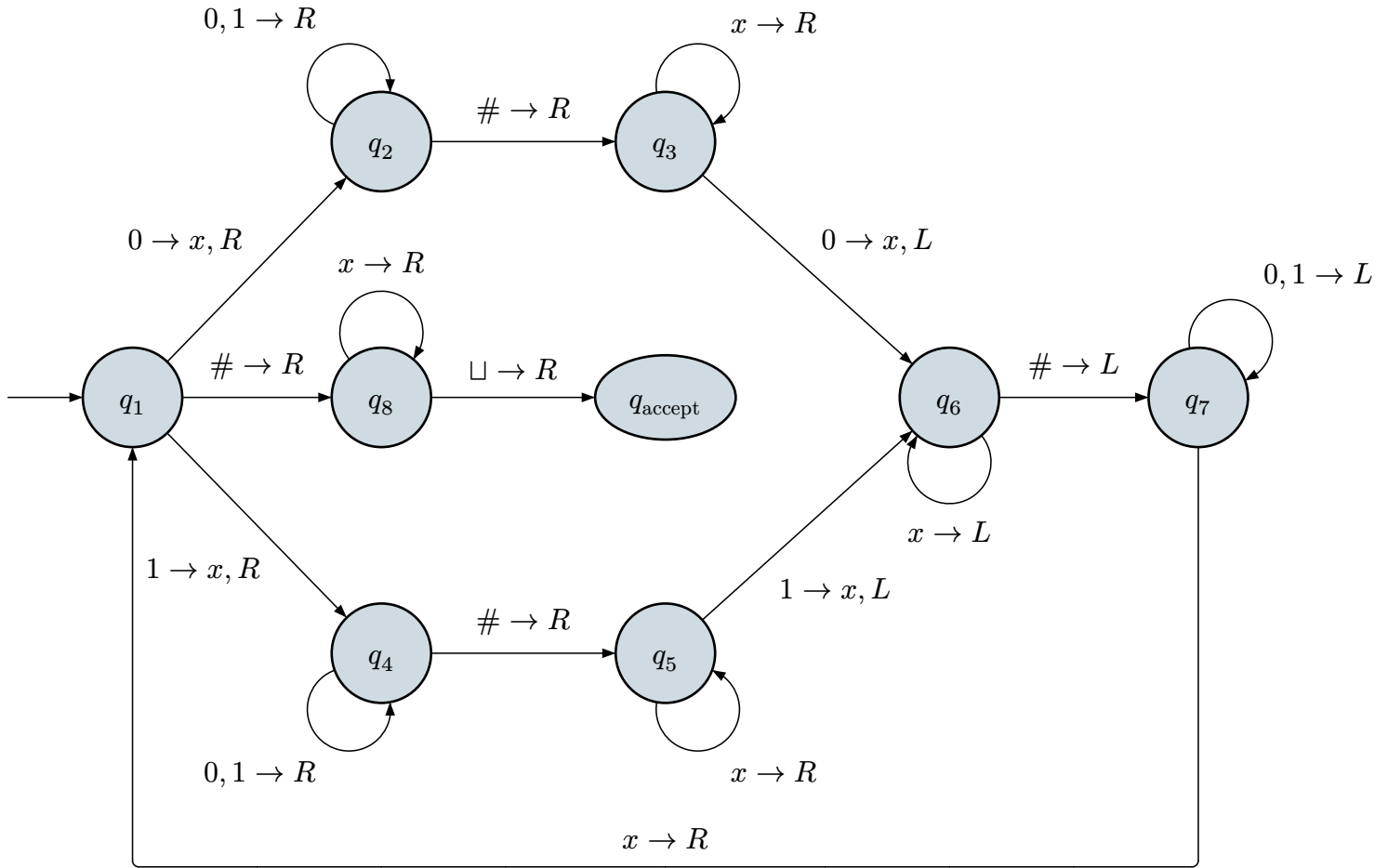


Figure 3.2: Turing Machine that decides the language $\{w\#w \mid w \in \{0,1\}^*\}$

3.5. Design and formally describe a Turing machine that decides $B = \{0^{2^n} \mid n \geq 0\}$

3.5.1. High Level description

The Turing machine will decide the language $B = \{0^{2^n} \mid n \geq 0\}$ by checking if the input string consists of a number of 0s that is a power of 2. The machine will:

1. Start at the leftmost cell of the tape.
2. cross off the first 0 by replacing it with an x.
3. Move the head to the right, and leave that 0 where it is.
4. Cross off the next 0 by replacing it with an x.
5. repeat the process until an empty cell is reached. If the machine just crossed off the last 0, then it will reject the input. Otherwise, the machine will move the head to the leftmost cell and repeat the process.

3.5.2. Formal description

Let $M = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$ be a Turing machine where:

- $Q = \{q_1, q_2, q_3, q_4, q_5, q_{\text{accept}}, q_{\text{reject}}\}$ is the set of states
- $\Sigma = \{0, \sqcup\}$ is the input alphabet
- $\Gamma = \{0, x, \sqcup\}$ is the tape alphabet
- start state: q_1

- Accept state: q_{accept}
- Reject state: q_{reject}

The transition function δ is defined by the following state diagram:

3.5.3. State Diagram

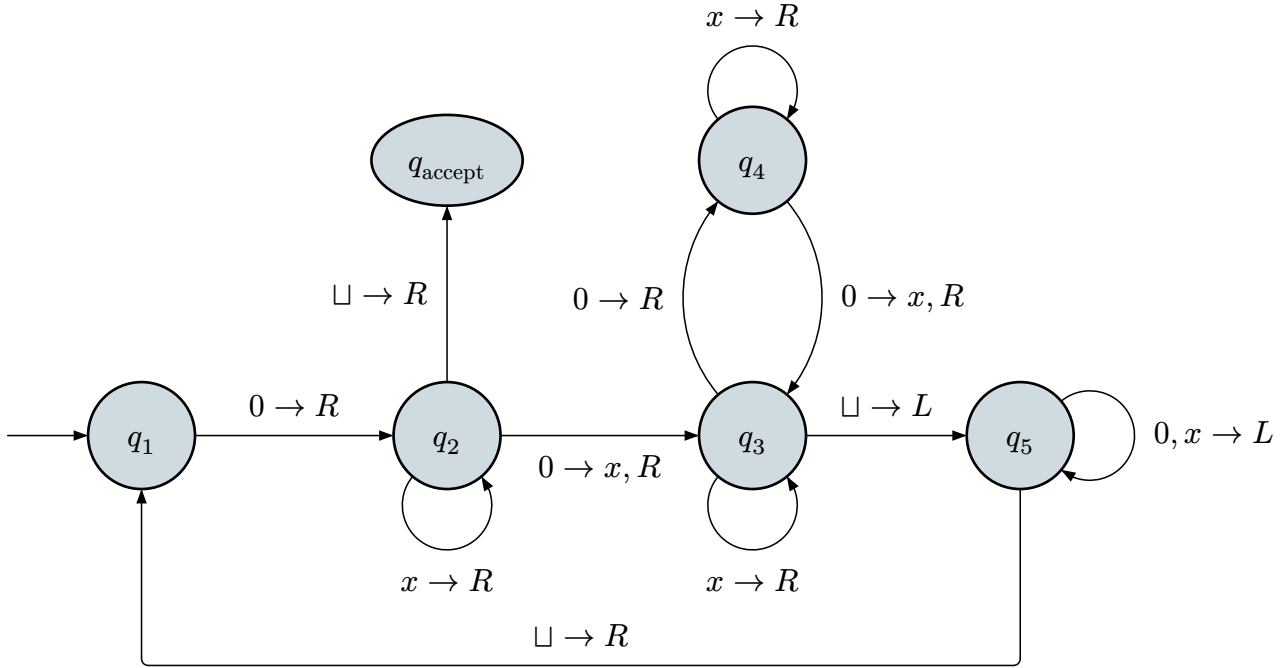


Figure 3.3: Turing Machine that decides the language $B = \{0^{2^n} \mid n \geq 0\}$

Note that any transition that is not explicitly defined in the state diagram is assumed to lead to the reject state q_{reject} .

3.5.4. A sample run of the machine on the input 0000

- | | |
|--------------------------------|--|
| 1. $q_1 0000$ | 14. $\square 0xxq_3x \square$ |
| 2. $0q_2000$ | 15. $\square 0xxxq_3 \square$ |
| 3. $0xq_300$ | 16. $\square 0xxq_5x \square$ |
| 4. $0x0q_40$ | 17. $\square 0xq_5xx \square$ |
| 5. $0x0xq_3 \square$ | 18. $\square 0q_5xxx \square$ |
| 6. $0x0q_5x \square$ | 19. $\square q_50xxx \square$ |
| 7. $0xq_50x \square$ | 20. $q_5 \square 0xxx \square$ |
| 8. $0q_5x0x \square$ | 21. $\square q_10xxx \square$ |
| 9. $\square q_50x0x \square$ | 22. $\square 0q_2xxx \square$ |
| 10. $q_5 \square 0x0x \square$ | 23. $\square 0xq_2xx \square$ |
| 11. $\square q_10x0x \square$ | 24. $\square 0xxq_2x \square$ |
| 12. $\square 0q_2x0x \square$ | 25. $\square 0xxxq_2 \square$ |
| 13. $\square 0xq_20x \square$ | 26. $\square 0xxx \square q_{\text{accept}}$ |

3.6. Design a Turing Machine that accepts the language

$$C = \{a^i b^j c^k \mid i \times j = k \wedge i, j, k \geq 1\}$$

3.6.1. High Level description

1. If the input doesn't match a^*b^*c after a full initial sweep, reject.
2. Move the head back to the leftmost cell.
3. Cross off an a with an x , scan to the right until a b is found.
4. cross off a b with a y , scan to the right until a c is found.
5. Cross off a c with a z , and move the head back to the leftmost uncrossed B
6. Repeat steps 4 and 5 until all b 's are crossed off.
- If all the c 's get crossed off, and there are still uncrossed a 's or b 's, then reject.
7. Move left until reach the leftmost uncrossed a , while uncrossing all the b 's.

3.6.2. Formal description

Let $M = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$ be a Turing machine where:

- $Q = \{q_1, q_2, q_3, q_4, q_5, q_6, q_{\text{accept}}, q_{\text{reject}}\}$ is the set of states
- $\Sigma = \{a, b, c\}$ is the input alphabet
- $\Gamma = \{a, b, c, x, y, z\}$ is the tape alphabet
- start state: q_1
- Accept state: q_{accept}
- Reject state: q_{reject}

The transition function δ is defined by the following state diagram:

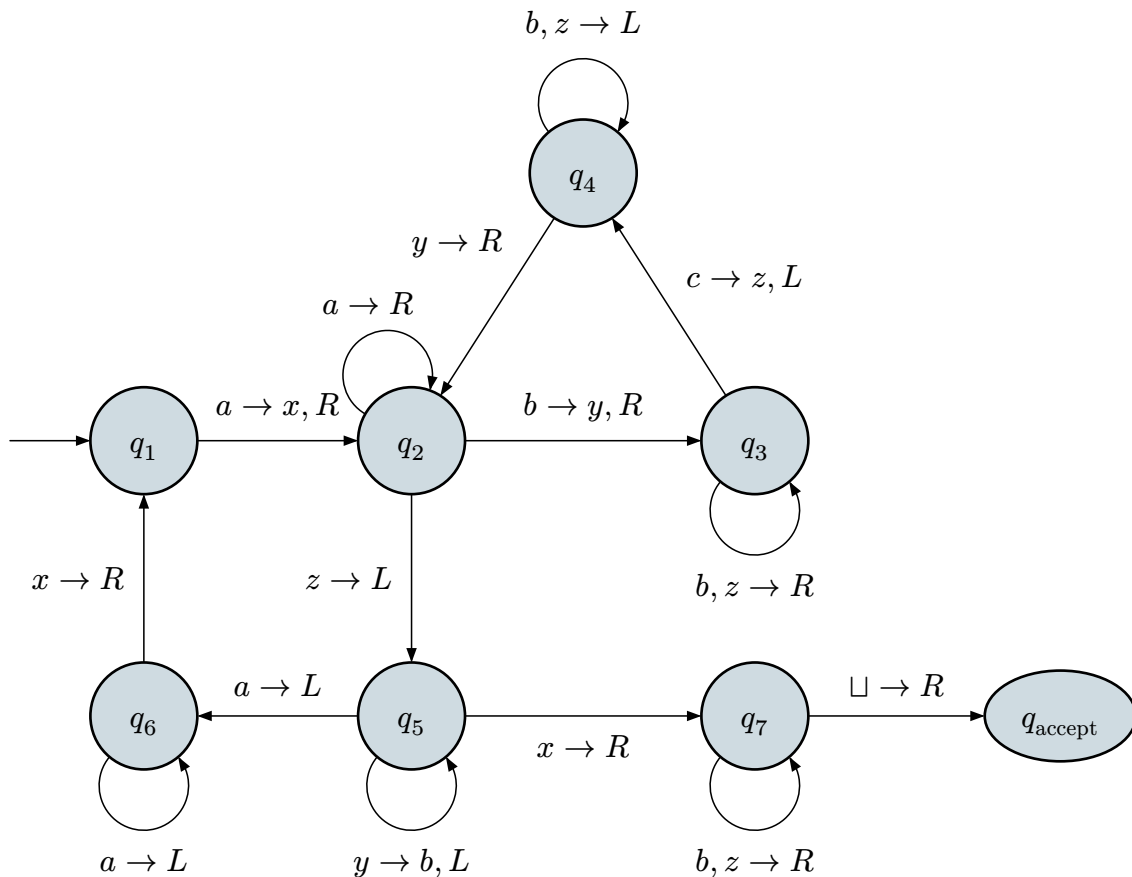


Figure 3.4: Turing Machine that decides the language $C = \{a^i b^j c^k \mid i \times j = k \wedge i, j, k \geq 1\}$

3.7. Review the formal definition of a Turing machine to answer the following questions, and explain your reasoning.

3.7.1. Can a Turing machine ever write the blank symbol $\sqcup$ on its tape?

Yes, a Turing machine can write the blank symbol $\sqcup$ on its tape. The blank symbol is part of the tape alphabet Γ , which is defined as the set of symbols that the Turing machine can write on the tape. The blank symbol is typically used to represent empty cells on the tape, and it is essential for the Turing machine's operation, especially when it needs to indicate that there are no more symbols to read or write in a particular direction.

3.7.2. Can the tape alphabet Γ be the same as the input alphabet Σ ?

No, the tape alphabet Γ cannot be the same as the input alphabet Σ . The tape alphabet must include all symbols from the input alphabet, but it can also include additional symbols such as the blank symbol $\sqcup$ and any other symbols that the Turing machine may need to write during its computation. The input alphabet is a subset of the tape alphabet.

3.7.3. Can a Turing machine's head ever be in the same location in two successive steps?

No, as a direction must be specified accordingly to many definitions, however some definitions of Turing machines allow the head to stay in the same location in two successive steps.

3.7.4. Can a Turing machine contain just a single state?

No, as we define the start state and the accept state to be 2 different states.

3.8. Discuss two proposed hypercomputation models, and how they avoid the limitations of Turing machines.

You should know the Church Turing Thesis!

Church-Turing thesis: any real-world computation can be translated into an equivalent computation involving a Turing machine (or represented using lambda calc). There are conflicting points of view about the Church-Turing thesis. One says that it can be proven, and other says that it serves as a definition for computation. There has never been a proof, but the evidence for its validity comes from the fact that every realistic model of computation, yet discovered, has been shown to be equivalent. If there were a device which could answer questions beyond those that a Turing machine can answer, then it would be called an oracle.

The rest of this stuff you don't need to know

1. Oracle machines: An oracle machine is a Turing machine augmented with an "oracle"—a black-box that can solve decision problems that are undecidable for ordinary Turing

machines, such as the Halting Problem. How it avoids Turing limits: Oracle machines bypass the limits of Turing machines by offloading undecidable tasks to the oracle, allowing them to compute problems beyond the Turing limit.

Don't necessarily violate thesis directly, as thesis applies to "physically realizable" computations. But expand the notion of what's considered "physically realizable" beyond classical digital Turing model.

1. Neural networks: Can learn and approximate complex functions by adjusting weights/biases, potentially computing non-computable functions. Weights of the connections between the nodes determines how much influence each input has on the output. Rely on training data to improve accuracy.

While Artificial neural networks don't directly violate the CT-thesis, they challenge traditional interpretations of computability by demonstrating that effective computation can occur through mechanisms that diverge from the strict sequential symbol manipulation of Turing machines:

- Parallel and distributed processing: ANNs perform computation through parallel and distributed processing, with interconnected neurons processing information simultaneously. This contrasts with the sequential execution of instruction in Turing machines.
- Some research areas explore how neural networks, especially those with features like chaotic dynamics or unbounded precision (e.g., recurrent neural networks (RNNs)), may be capable of hypercomputation. However, the formal proof of hypercomputation by neural networks remains a challenge and is not yet widely accepted. At the end of the day, ANNs do not directly violate the thesis because their computations are ultimately representable and able to be simulated by a TM.

3.9. The Acceptance Problem

The acceptance problem is a decision problem that asks whether a given Turing machine accepts a specific input. It is formally defined as follows:

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts input } w \} \quad (3.1)$$

Essentially, A is the set of encoded pairs of Turing machines and inputs for which the Turing machine accepts the input. As it is encoded, it is a language, and as such, if we can find a Turing machine that accepts this language, then we can say that the acceptance problem is decidable. (As if we can design a Turing machine that accepts the language, then we can say that the acceptance problem is decidable).

3.9.1. Is the acceptance problem recognisable?

Refresher: A language is recognisable if there exists a Turing machine that accepts all strings in the language and either rejects or loops on strings not in the language. Yes, the acceptance problem is recognisable. We can construct a Turing machine R that recognizes the acceptance problem by simulating the Turing machine M on the input w . If M accepts w , R will accept; if M rejects w or loops indefinitely, R will either reject or loop indefinitely as well. Thus, R recognizes the language A_{TM} .

3.9.2. Is the acceptance problem decidable?

Proof by contradiction: Let us assume that the acceptance problem is decidable. This means that there exists a Turing machine H that can decide whether any Turing machine M accepts a given input w . We can construct a new Turing machine H as follows:

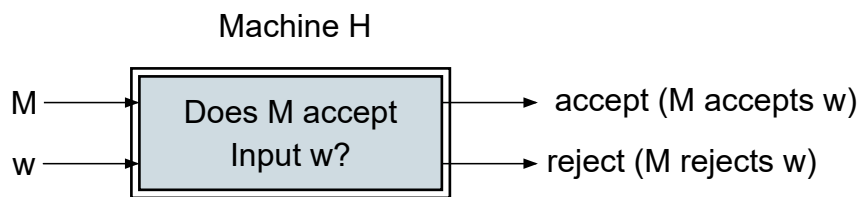


Figure 3.5: Turing Machine H that decides the acceptance problem

Now, let's be clever and design another Turing machine D which takes the input M , and does the following:

- It runs H on the inputs M and M (encoded as an input).
 - This is essentially asking whether M accepts its own encoding as input.
- If H accepts, D rejects.
- If H rejects, D accepts.

Let's see how the diagram looks:

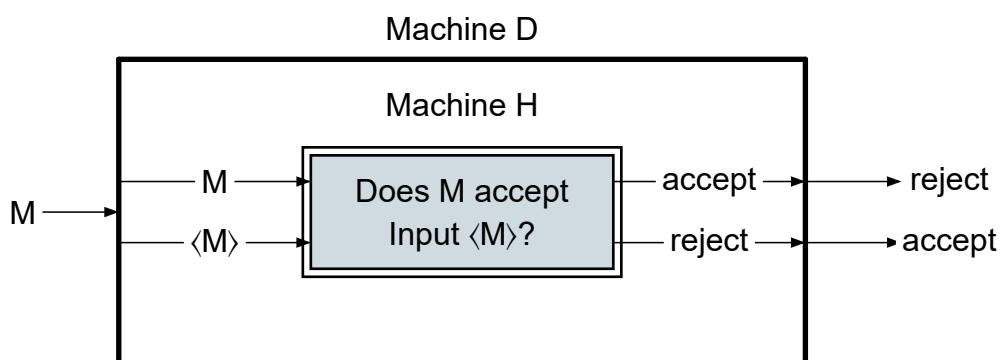


Figure 3.6: Turing Machine D that decides the acceptance problem

Now the final step, we can feed D back into itself as input, i.e. we run D on the input $\langle D \rangle$.

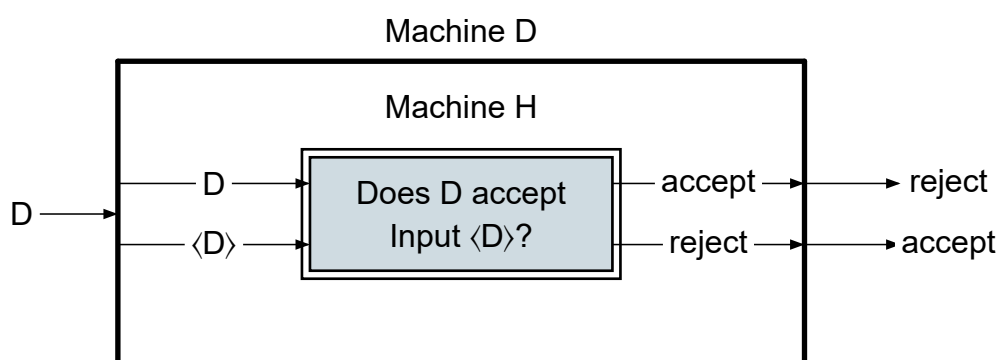


Figure 3.7: Turing Machine D that decides the acceptance problem

So what happens if machine D accepts its own encoding?

- If D accepts its own encoding, then it means that Machine H rejects the input of D , $\langle D \rangle$, which means that D does not halt on its own encoding. But this is a contradiction, as we assumed that D accepts its own encoding in this scenario.

- If D rejects its own encoding, then it means that Machine H accepts the input of D, $\langle D \rangle$, which means that D halts on its own encoding. But this is a contradiction, as we assumed that D rejects its own encoding in this scenario.

Thus, we have reached a contradiction in both cases, which means that our initial assumption that the acceptance problem is decidable is false. Therefore, the acceptance problem is undecidable.

3.10. The Halting Problem Revisited

The Halting Problem is a well-known undecidable problem in computability theory. It asks whether a given Turing machine M will halt (i.e., stop running) on a given input w . The formal definition of the Halting Problem is:

$$H = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ halts on input } w \} \quad (3.2)$$

Using the fact that we just showed that the acceptance problem is undecidable, we can show that the Halting Problem is also undecidable. This sort of proof is called a **Reduction** proof.

The basic idea:

1. Supposed that the Halting Problem is decidable, i.e., there exists a Turing machine H that can decide whether any Turing machine M halts on a given input w .
2. We can use H to construct a new Turing machine D that decides the acceptance problem.
3. But we already showed that the acceptance problem is undecidable
4. Therefore, our assumption that H exists must be false, and the Halting Problem is undecidable.

3.10.1. The Formal Proof

Let us assume that the Halting Problem is decidable. This means that there exists a Turing machine H that can decide whether any Turing machine M halts on a given input w .

$$H(\langle M, w \rangle) = \begin{cases} \text{accept if } M \text{ halts on } w \\ \text{reject if } M \text{ loops on } w \end{cases} \quad (3.3)$$

We can use H as a subroutine to construct a new Turing machine D that decides the acceptance problem. The machine D will work as follows:

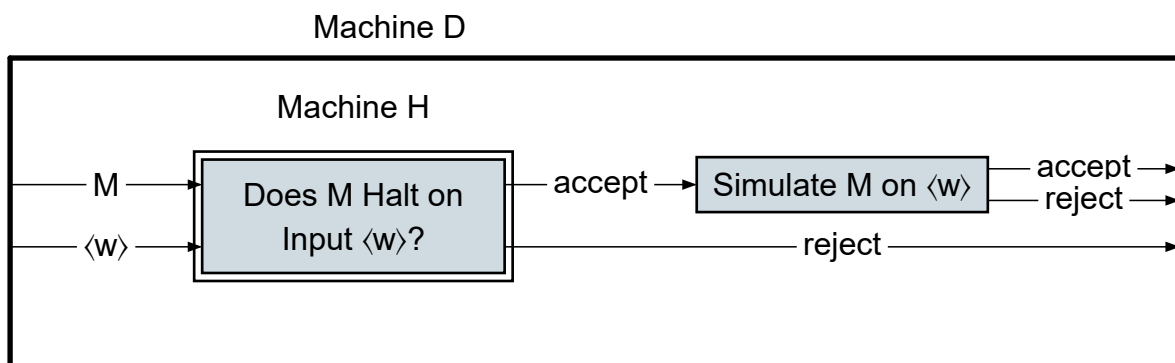


Figure 3.8: Turing Machine D that decides the acceptance problem

1. D first calls H with the input (M, w).
2. If H rejects, then D rejects as well – As if M loops, then we know it does not accept w.
3. If H accepts, then D simulates M on w.
 - If M halts and accepts w, then D accepts.
 - If M halts and rejects w, then D rejects.

Thus, D decides the acceptance problem, which we already showed is undecidable. Therefore, our assumption that the Halting Problem is decidable must be false, and the Halting Problem is undecidable.

3.11. Validity and Significance of the Church-Turing thesis

- Remains a foundational principle in computer science, guiding the understanding of computability and the limits of algorithmic processes.
- However, we may need to reconsider the thesis in light of new computational paradigms, such as quantum computing, neural networks, and other emerging technologies that challenge traditional notions of computation.
- We can view it as applying to classical computers, and allowing alternative models under different physical constraints.
- We shouldn't discard it lightly – as it has enabled major progress in computability/complexity theory.

3.12. Can a Turing machine obtain Artificial Intelligence?

If we are referring to general AI, then no (according to Shakes). Many arguments for and against this exist. Roger Penrose argues that the human mind is not computable, and that consciousness cannot be reduced to a Turing machine. As such, the only way to completely simulate human intelligence is to completely simulate the human brain, (in other words, make another human). As such, our current understanding of Turing machines and computability does not allow for the creation of a true AI that can replicate human-like intelligence or consciousness.

However, if we are referring to LLM's (Large Language Models) or other AI systems, then yes, they can be implemented as Turing machines. LLMs are trained on vast amounts of data and can generate human-like text based on patterns learned during training. They operate within the framework of Turing machines, processing input and generating output based on learned algorithms and models. However, they do not possess true understanding or consciousness, as they are fundamentally statistical models rather than sentient beings.

However, it is interesting to note that if you ask a LLM whether it has consciousness, it will say that it is confused, and raises an interesting question regarding how much information processing is required to be considered conscious? Are they the same thing?

4. Von Neumann Architecture

4.1. Background

What is the connection between TMs and Von Neumann's architecture? Alan Turing and John von Neumann laid the foundations for modern computing in the mid-20th century. The whole idea of the Turing device was to solve a problem in mathematical logic completely disconnected with real-life applications. Turing's theoretical concept of a universal machine provided the basis for contemporary computing architecture; Turing's machine was not intended to be a practical computing device but rather a mathematical model to explore the limits and capabilities of computation. It served as a foundation for the development of modern computers by providing a framework for understanding the fundamental principles of algorithms, automata, and computability. At the same time, von Neumann proposed the stored-program architecture, the blueprint for nearly every computer built since then. The stored-program concept refers to a computer's ability to store both data and instructions in the same memory. This allows for the execution of complex algorithms by enabling the computer to fetch instructions from its memory, execute them, and then store the results back into memory for further processing. This concept is essential for the flexibility and versatility of modern computers

4.2. Von Neumann Architecture

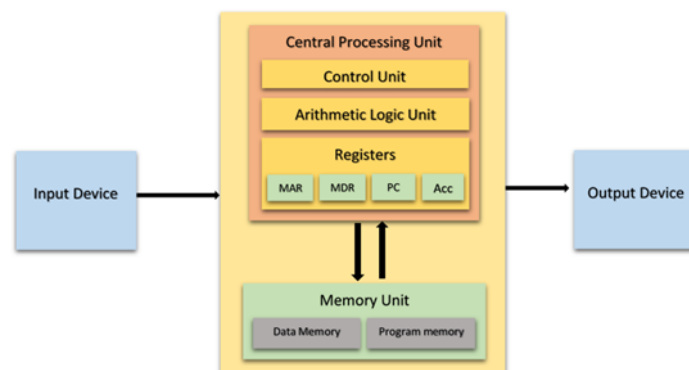


Figure 4.1: Von Neumann Architecture

Registers are primarily found in the CPU. The CPU houses a variety of registers, which are high-speed storage areas that hold data and instructions used by the processor; These registers hold data and instructions needed during the Fetch-Decode-Execute cycle.

Both instruction data and program data are stored in the same memory. This is done in binary form (which hard to understand for people so when you see examples online of the architecture showing the memory contents, people write it in more human-readable writing (example below)).

Fetch:

- The program counter is checked as it holds the address of the next instruction to be executed.
- This address is used by the MAR in order to fetch the instruction needed from RAM and bring it into MDR.
- The address in the program counter is incremented to point at the next instruction.

Decode:

- Now the instruction is in the CPU, the CU decodes the instruction to see what has to be done.

Execute:

- Now we know what to do, we can execute the instruction.
- What we actually do depends on what the instruction is. For example:
 - We could be asked to head back to RAM and fetch some data and add it to the accumulator
 - We could be asked to jump to another instruction.
 - We could be asked to write data held in the accumulator back into RAM.

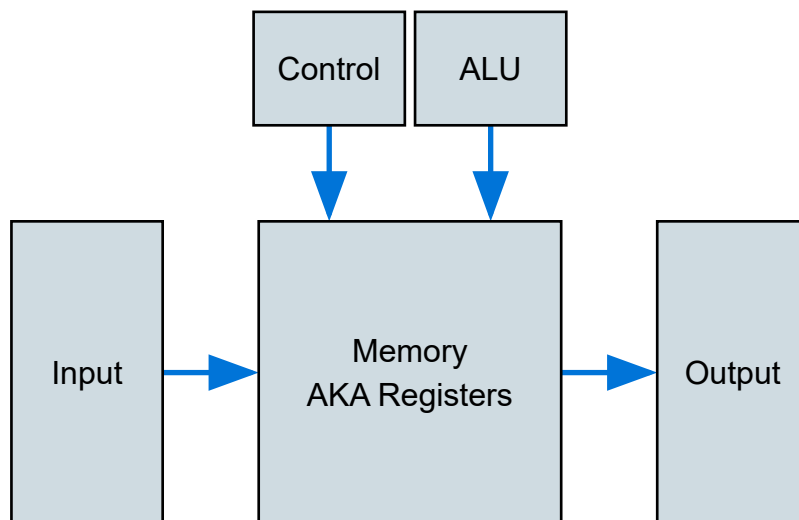


Figure 4.2: Von Neumann Architecture Components

The CPU is one of the most important components in the von Neumann architecture and consists of an ALU and registers to store the current intermediate input and outputs. Typically, these registers are faster, but smaller memory components closer to the ALU for rapid access. The ALU consists of arithmetic and logic circuits capable of adding, subtracting, multiplying integers and computing logical operations such as AND, OR and XOR etc. of inputs stored within the registers. The output of the ALU is stored in another register and can be copied to another register for further computation or to the main memory for later use. The registers and ALU are connected to each other through a series of buses, which are circuits designed to carry signals at high speeds. For example, in a typical 'cycle' of a CPU, an instruction fetches two inputs (operands for either arithmetic or logic operations) from the registers to the ALUs input registers, performs the desired operation (say addition) with the ALU and stores the result in an output register. This cycle and the structure of the units involved determine what the computer can do.

There are 4 types of registers in the CPU:

- **Program Counter (PC):** Holds the address of the next instruction to be executed.
- **Memory Address Register (MAR):** Holds the address of the memory location to be accessed.
- **Memory Data Register (MDR):** Holds the data being transferred to or from memory.
- **Accumulator (ACC):** Holds the intermediate results of arithmetic and logic operations.

The registers interact with the memory (RAM) through the memory bus, which is a set of wires that carry data between the CPU and memory. The CPU uses the MAR to specify the address in memory where data is to be read or written, and the MDR to hold the data being transferred.

4.3. Similarities and Differences between Turing Machines and Von Neumann Architecture

Tape:

- The Turing machine's infinite tape, used for storing data and instructions, is directly mirrored in the memory of a modern computer. This memory can be RAM, ROM, or other forms of storage, but the principle remains the same: a place to store data and programs that the machine can access.

Read/Write Head:

- The Turing machine's read/write head reads from and writes to the tape. In a modern computer, this function is performed by the Central Processing Unit (CPU). The CPU reads instructions and data from memory, processes them, and then writes the results back to memory.

Finite State Control:

- The Turing machine's finite state control dictates the machine's behaviour based on its current state, input symbol, and a set of rules. This is analogous to the program code in a modern computer. The CPU follows the instructions in memory, executing them one after another, and the current state of the machine is represented by the program counter and registers.

Start state of computation:

- Program Counter (PC) set to address of first instruction

Accept (halt-success) state:

- End-of-program or success return in programs (e.g., return code 0)

Reject (halt-failure) state:

- Error-handling or crash state (e.g., exceptions, return code $\neq 0$)

Changes:

- For TMs, δ is external to the machine; the rules are hardcoded by design. A TM's "instruction" is its transition function, a table that dictates the TM's behaviour based on its current state and the symbol it's reading. This function determines the next state, the symbol to write, and the direction the head moves (left or right). For VNA, Programs (δ -like instructions) are stored in memory along with data, and the CPU deals with those programs; Von Neumann architecture processes instructions from memory, where each instruction specifies an operation to be performed. The ISA defines the machine's instruction set. The CPU microarchitecture implements the behaviour specified by instructions.

- The concept of a stored program that could be kept directly in the memory for ease of retrieval and updating during the computation or algorithm. The design of the CPU consisting of registers, arithmetic and logic units we still use today.
- The use of binary arithmetic was to simplify the hardware from storing decimal numbers (0-9) in 10 vacuum tubes to electronics with zeros and ones instead.
- Has a linear, addressable memory vs TM's infinite tape. Additionally, the memory is a unified structure for both instructions and data.

TM equivalents (if possible) (not necessarily 1-to-1) to CPU components:

- ALU: In TMs, there is no dedicated arithmetic component (vs VNA that has the ALU). In TMs, the computation is performed through state transitions and symbol manipulations. Mathematical operations must be encoded entirely in the transition function (δ). Each arithmetic operation requires many state transitions and tape movements.
- CU: The control unit is characterized by a finite set Q of control states, a start state $q_0 \in Q$ and a transition function δ .
- Register equivalent: no equivalent. If you really want to force an answer, in TMs, essentially the only 'memory' is the idea of what state we are currently in.

Extra:

1. **Integration vs. Separation:**

- TM combines all control logic into a single transition function
- Von Neumann architecture separates concerns into specialized components

2. **Data Processing:**

- TM requires multiple state transitions for even simple operations
- CPUs perform complex operations in single instructions using dedicated hardware

3. **Abstraction Level:**

- TM is a mathematical abstraction focused on computability theory
- Modern CPU is an engineering solution focused on practical performance

5. Lambda Calculus

Lambda calculus is a model of computation that is equivalent to Turing machines (as per the Church-Turing thesis). Lambda calculus reduces all computation to the basic operations involving only functions and their applications.

5.1. Syntax

Church introduced the λ notation to functions to allow long strings of functions to be chained or composed together that is both minimal and compact. We remove the parentheses and equal signs to a traditional function because they are implied, introduce the λ symbol to denote the beginning of a function and use the period symbol to denote the chaining or composition of all elements.

The syntax of lambda calculus consists of:

- Variables: $x, y, z, \dots$
- Abstractions: $\lambda x.E$, where E is an expression
- Applications: $E_1 E_2$, where E_1 and E_2 are expressions
- Parentheses: Used to group expressions and clarify order of evaluation

5.2. Identity Function

We can define any functions that we want to in lambda calculus. The easiest to observe is the identity function, which is defined as:

$$T := \lambda x.x \quad (5.1)$$

The $:=$ operator is used whenever we want to define a new function. λ denotes function definition with arguments and the symbols after the period as the body of the function

When we apply the identity function to an argument, it simply returns that argument:

$$Ta = (\lambda x.x)a = a \quad (5.2)$$

Note that the way this happened is that we substituted the argument (a) for every instance of x within all the parentheses, and then removed the lambda. So applying it with a numerical value, say 5, would yield:

$$T5 = (\lambda x.x)5 = 5 \quad (5.3)$$

In future, we will not be naming the functions, but rather will use their definition directly in the expression.

5.3. Handling Multiple Arguments

In lambda calculus, we can handle multiple arguments by using nested abstractions. For example, a function that takes two arguments can be defined as:

$$\lambda x.(\lambda y.x + y) \quad (5.4)$$

This function takes an argument x and returns another function that takes an argument y and computes $x + y$. When we apply this function to two arguments, we can do it as follows:

$$(\lambda x.(\lambda y.x + y))ab = (\lambda y.a + y)b = a + b \quad (5.5)$$

See below where we supply the function with the arguments 3, and 5:

$$(\lambda x.(\lambda y.x + y))35 = (\lambda y.3 + y)5 = 3 + 5 = 8 \quad (5.6)$$

5.4. Beta Reductions

Lambda expressions are evaluated from left to right, and the process of applying a function to an argument is called beta reduction. Beta reduction involves substituting the argument for the bound variable in the body of the function. Let's see a concrete example of beta reduction:

$$(\lambda x.(\lambda y.x + y)3)2 = (\lambda y.2 + y)3 = 2 + 3 = 5 \quad (5.7)$$

This process doesn't require any numerical values either, we can also use variables:

$$(\lambda x.xx)(\lambda y.y) = (\lambda y.y)(\lambda y.y) \quad (5.8)$$

Here's another example:

$$(\lambda x.x(\lambda x.x))y = y(\lambda x.x) \quad (5.9)$$

5.5. Church Encoding

The rest of lambda calculus will not utilise any other functions other than functions which are explicitly defined in terms of other pure lambda functions. This is known as Church encoding, and it allows us to represent data structures and operations purely in terms of lambda calculus.

5.5.1. Church Numerals

We can represent natural numbers in lambda calculus using Church numerals. A church numeral represents the natural number n as a function that applies another function f exactly n times. As an example, the Church numeral for 1 is defined as:

$$1 := \lambda f.\lambda x.f(x) \quad (5.10)$$

The Church numeral for 2 is defined as:

$$2 := \lambda f.\lambda x.f(f(x)) \quad (5.11)$$

And so on and so forth. For 0, we define it as:

$$0 := \lambda f.\lambda x.x \quad (5.12)$$

Note that we apply the function f zero times, so we just return the initial value x .

5.5.2. Successor Function

We can define the successor function in lambda calculus, which takes a Church numeral and returns the next Church numeral. The successor function is defined as:

$$S := \lambda n.\lambda y.\lambda x.y(nyx) \quad (5.13)$$

This function takes a Church numeral n and returns a new Church numeral that applies the function f one more time than n does. For example, applying the successor function to the Church numeral for 0 gives us the Church numeral for 1:

$$\begin{aligned}
S0 &= (\lambda n. \lambda y. \lambda x. y(nyx))(\lambda f. \lambda x. x) \\
&= (\lambda y. \lambda x. y((\lambda f. \lambda x. x)yx)) \\
&= (\lambda y. \lambda x. y((\lambda x. x)x)) \\
&= (\lambda y. \lambda x. y(x))
\end{aligned} \tag{5.14}$$

Which has the exact form as the Church numeral for 1. Note that even though the variable y is not used in our Church numeral for 1, it still is the same dummy variable, and thus the output is the same.

To further illustrate the successor function, let's apply it to the Church numeral for 2:

$$\begin{aligned}
S2 &= (\lambda n. \lambda y. \lambda x. y(nyx))(\lambda f. \lambda x. f(f(x))) \\
&= (\lambda y. \lambda x. y((\lambda f. \lambda x. f(f(x)))yx)) \\
&= (\lambda y. \lambda x. y((\lambda x. y(y(x)))x)) \\
&= (\lambda y. \lambda x. y(y(y(x))))
\end{aligned} \tag{5.15}$$

Which has the exact form as the Church numeral for 3.

5.5.3. Church Arithmetic

IMPORTANT – THIS IS NOT ASSESSED IN THE FINAL ONLY READ IF INTERESTED

Multiplication: We can define multiplication in lambda calculus using Church numerals. The multiplication function takes two Church numerals and returns their product as a Church numeral. It is defined as:

$$M := \lambda n. \lambda k. \lambda f. \lambda x. n(kf) \tag{5.16}$$

Lets see it in action when we multiply 2 and 3:

$$\begin{aligned}
M\mathbb{N}_2\mathbb{N}_3 &= (\lambda n. \lambda k. \lambda f. \lambda x. n(kf))(\lambda f. \lambda x. f(f(x)))(\lambda f. \lambda x. f(f(f(x)))) \\
&= (\lambda k. \lambda f. \lambda x. (\lambda f. \lambda x. f(f(x)))(kf))(\lambda f. \lambda x. f(f(f(x)))) \\
&= (\lambda f. \lambda x. (\lambda f. \lambda x. f(f(x)))(f(f(f(x))))f) \\
&= (\lambda f. \lambda x. (\lambda f. \lambda x. f(f(x)))(\lambda x. f(f(f(x)))))) \\
&= (\lambda f. \lambda x. (\lambda x. f(f(f(x))))((\lambda x. f(f(f(x))))(x))) \\
&= (\lambda f. \lambda x. (\lambda x. f(f(f(x))))(f(f(f(x)))))) \\
&= (\lambda f. \lambda x. (f(f(f(f(f(f(x))))))))
\end{aligned} \tag{5.17}$$

If we count the number of times we apply the function f , we see that it is 6 times, which is the product of 2 and 3. **Addition:** We can define addition in lambda calculus using Church numerals. The addition function takes two Church numerals and returns their sum as a Church numeral. It is defined as:

$$A := \lambda n. \lambda k. \lambda f. \lambda x. nf(kfx) \tag{5.18}$$

Let's see it in action when we add 2 and 2:

$$A\mathbb{N}_2\mathbb{N}_2 = (\lambda n.\lambda k.\lambda f.\lambda x.nf(kfx)) (\lambda f.\lambda x.f(fx)) (\lambda f.\lambda x.f(fx)) \quad (5.19)$$

$$= (\lambda k.\lambda f.\lambda x.(\lambda f.\lambda x.f(fx))f(kfx)) (\lambda f.\lambda x.f(fx)) \quad (5.20)$$

$$= \lambda f.\lambda x.(\lambda f.\lambda x.f(fx))f((\lambda f.\lambda x.f(fx))fx) \quad (5.21)$$

$$= \lambda f.\lambda x.(\lambda x.f(fx))((\lambda f.\lambda x.f(fx))fx) \quad (5.22)$$

$$= \lambda f.\lambda x.(\lambda x.f(fx))((\lambda x.f(fx))x) \quad (5.23)$$

$$= \lambda f.\lambda x.(\lambda x.f(fx))(f(fx)) \quad (5.24)$$

$$= \lambda f.\lambda x.f(f(f(fx))) \quad (5.25)$$

This shows that we apply the function f 4 times, which is the sum of 2 and 2.

5.6. Boolean Algebra

In lambda calculus, we can represent boolean values and operations using Church encoding. The boolean values true and false can be defined as:

$$\mathbb{T} := \lambda x.\lambda y.x \quad (\text{represents true}) \quad (5.26)$$

$$\mathbb{F} := \lambda x.\lambda y.y \quad (\text{represents false}) \quad (5.27)$$

The boolean operations can be defined as follows:

Not: The not operation takes a boolean value and returns its negation. It is defined as:

$$\text{NOT} := \lambda b.b\mathbb{F}\mathbb{T} \quad (5.28)$$

Let's see it in action when we apply it to true:

$$\text{NOT } \mathbb{T} = (\lambda b.b\mathbb{F}\mathbb{T})\mathbb{T} = \mathbb{T}\mathbb{F}\mathbb{T} = \mathbb{F} \quad (5.29)$$

Let's see it in action when we apply it to false:

$$\text{NOT } \mathbb{F} = (\lambda b.b\mathbb{F}\mathbb{T})\mathbb{F} = \mathbb{F}\mathbb{F}\mathbb{T} = \mathbb{T} \quad (5.30)$$

And: The and operation takes two boolean values and returns true if both are true, otherwise false. It is defined as:

$$\text{AND} := \lambda p.\lambda q.pqp \quad (5.31)$$

Let's see it in action when we apply it to true and false:

$$\text{AND } \mathbb{T}\mathbb{F} = (\lambda p.\lambda q.pqp)\mathbb{T}\mathbb{F} = \mathbb{T}\mathbb{F}\mathbb{T} = \mathbb{F} \quad (5.32)$$

Let's see it in action when we apply it to true and true:

$$\text{AND } \mathbb{T}\mathbb{T} = (\lambda p.\lambda q.pqp)\mathbb{T}\mathbb{T} = \mathbb{T}\mathbb{T}\mathbb{T} = \mathbb{T} \quad (5.33)$$

Or: The or operation takes two boolean values and returns true if at least one is true, otherwise false. It is defined as:

$$\text{OR} := \lambda p.\lambda q.ppq \quad (5.34)$$

Let's see it in action when we apply it to true and false:

$$\text{OR } \mathbb{T}\mathbb{F} = (\lambda p.\lambda q.ppq)\mathbb{T}\mathbb{F} = \mathbb{T}\mathbb{T}\mathbb{F} = \mathbb{T} \quad (5.35)$$

Let's see it in action when we apply it to false and false:

$$\text{OR } \mathbb{F}\mathbb{F} = (\lambda p.\lambda q.ppq)\mathbb{F}\mathbb{F} = \mathbb{F}\mathbb{F}\mathbb{F} = \mathbb{F} \quad (5.36)$$

5.7. Combinators

Combinators are higher-order functions that can be used to manipulate other functions. They are a key part of lambda calculus and can be used to define complex operations without the need for variables, and do not have free variables themselves. Essentially, they are “Macros” in lambda calculus that can be used to build more complex functions.

The first combinator we will look at is the **K combinator**, which takes two arguments and returns the first one, ignoring the second. It is defined as:

$$K := \lambda x.\lambda y.x \quad (5.37)$$

Note that this is the same as the boolean value true in Church encoding, as it takes two arguments and returns the first one. Let's see how we would define the boolean value false using the K combinator:

$$\mathbb{F} := KI \quad (5.38)$$

where I is the identity function. As such, we can test this by applying it to two arguments:

$$\mathbb{F}ab = KIab = Ib = b \quad (5.39)$$

This works as the K combinator grabs the first argument (which is the identity function). So $KIa = I$. Thus, we are left with the third argument, which is b .

Another important combinator is the **C combinator**, which is the NOT function from above:

$$C := \lambda f.\lambda x.\lambda y.fyx \quad (5.40)$$

Let's see how we can use the C combinator to flip the K combinator:

$$\begin{aligned} CK &= (\lambda f.\lambda x.\lambda y.fyx)K \\ &= (\lambda x.\lambda y.Kyx) \\ &= (\lambda x.\lambda y.y) \\ &= \mathbb{F} \text{ or } KI \end{aligned} \quad (5.41)$$

5.8. The Y Combinator

The Y combinator is a fixed-point combinator in lambda calculus that allows us to define recursive functions. It is defined as:

$$Y := \lambda f.(\lambda x.f(xx))(\lambda x.f(xx)) \quad (5.42)$$

Let's see how it works with a given placeholder function g :

$$\begin{aligned} Yg &= (\lambda f.(\lambda x.f(xx))(\lambda x.f(xx)))g \\ &= (\lambda x.g(xx))(\lambda x.g(xx)) \\ &= g(\lambda x.g(xx))(\lambda x.g(xx)) \\ &= g(Yg) = g(g(Yg)) \text{ etc} \end{aligned} \quad (5.43)$$

The tricky part here is that for line 3, we are substituting the rightmost $\lambda x.g(xx)$ inside the other $\lambda x.g(xx)$, which is why a g appears on the outside of the parentheses. This allows us to define recursive functions in lambda calculus, as we can now refer to the function itself within its definition.

6. Quantum Computing

The notation that is used for classical bits is actually the same as the notation used for qubits. The only difference is that the qubit notation is not limited to 0 and 1. In other words, the classical bit notation is a subset of the qubit notation (or in other words, a special case).

A qubit is represented as follows:

$$|x\rangle = \begin{pmatrix} a \\ b \end{pmatrix} \quad \|a\|^2 + \|b\|^2 = 1 \quad (6.1)$$

where a and b are complex numbers. Some “complex” numbers are shown below:

$$\begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \quad \begin{pmatrix} \frac{1}{2} \\ \frac{\sqrt{3}}{2} \end{pmatrix} \quad \begin{pmatrix} -1 \\ 0 \end{pmatrix} \quad \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix} \quad (6.2)$$

(note that I haven’t written an imaginary component, as we don’t do that in 2048)

6.1. Superposition (Why Qubits are Cool)

The idea that a qubit can have a value which is neither 0 nor 1 is called superposition. Superposition actually means that the qubit is both 0 **and** 1 at the same time.

When you measure the qubit, it will collapse to either 0 or 1, it can’t become anything else. This is generally only done at the end of a circuit (when you want to read the result).

If a qubit has value $|x\rangle = \begin{pmatrix} a \\ b \end{pmatrix}$, then measuring the qubit will yield 0 with probability $\|a\|^2$ and 1 with probability $\|b\|^2$.

Applying this to our previous qubits we obtain the following probabilities:

Qubit	Probability of 0	Probability of 1
$\begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}$	$\left(\frac{1}{\sqrt{2}}\right)^2 = \frac{1}{2}$	$\left(\frac{1}{\sqrt{2}}\right)^2 = \frac{1}{2}$
$\begin{pmatrix} \frac{1}{2} \\ \frac{\sqrt{3}}{2} \end{pmatrix}$	$\left(\frac{1}{2}\right)^2 = \frac{1}{4}$	$\left(\frac{\sqrt{3}}{2}\right)^2 = \frac{3}{4}$
$\begin{pmatrix} -1 \\ 0 \end{pmatrix}$	$1^2 = 1$	$0^2 = 0$
$\begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}$	$\left(\frac{1}{\sqrt{2}}\right)^2 = \frac{1}{2}$	$\left(-\frac{1}{\sqrt{2}}\right)^2 = \frac{1}{2}$
$\begin{pmatrix} 1 \\ 0 \end{pmatrix}$	$1^2 = 1$	$0^2 = 0$
$\begin{pmatrix} 0 \\ 1 \end{pmatrix}$	$0^2 = 0$	$1^2 = 1$

Table 6.1: Probabilities of measuring a qubit

At the very end of the table, note that we have the classical bits 0 and 1. They are also qubits but their probabilities are certain (100%).

6.2. Tensor Product – The first Step to Qubits

We can represent classical bits using the tensor product. For example, if we have two classical bits, their representation is in the below table:

Classical Bits	Tensor Product	Matrix
$ 00\rangle$	$\begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$
$ 01\rangle$	$\begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix}$	$\begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$
$ 10\rangle$	$\begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$
$ 11\rangle$	$\begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$

Table 6.2: Representing multiple classical bits

The easiest way to understand the conversion of multiple classical bits to a matrix is to visualise the matrix as a 1 dimensional array. The value of the bit to be represented is the index of the array. For example, the classical bit $|01\rangle$ has a numerical value of 1 (converting from binary to decimal). Thus, the matrix representation of this bit has a 1 located at index 1, as can be seen above.

6.3. Representing Multiple Qubits

The representation of multiple qubits is the same as the representation of multiple classical bits. We obtain the same matrix representation as before by applying the tensor product to the qubits. Take the example below:

$$\begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \otimes \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{2} \end{pmatrix} \quad (6.3)$$

Note that if we square each of the elements in the matrix, we obtain the probabilities of measuring each of the qubits. In the case above, we have 4 qubits, and the probability of measuring each of the combinations ($|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$) is 1/4 each.

6.4. Qubit Operations

Operations on qubits are the same as operations on classical bits. For example, let's see how the NOT gate works on a qubit:

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{2} \\ \frac{\sqrt{3}}{2} \end{pmatrix} = \begin{pmatrix} \frac{\sqrt{3}}{2} \\ \frac{1}{2} \end{pmatrix} \quad (6.4)$$

As can be seen here, all the NOT gate has done is swap the probabilities of measuring 0 and 1.

6.4.1. Hadamard Gate

Certain gates only make sense in the quantum world. One of these is the Hadamard gate. The Hadamard gate takes a 0-bit or 1-bit and puts it into exactly equal superposition. As with all gates, the Hadamard gate has a matrix representation, which can be seen below:

$$H = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad (6.5)$$

The results of applying the Hadamard gate to a qubit are shown in the table below:

Qubit	Hadamard Gate	Result
$\begin{pmatrix} 1 \\ 0 \end{pmatrix}$	$H \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix}$	$\begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}$
$\begin{pmatrix} 0 \\ 1 \end{pmatrix}$	$H \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix}$	$\begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}$

Table 6.3: Hadamard gate on qubits

An important thing to note here is that even though the result qubits both have a 50-50% chance of being 0 or 1, they are actually two different states of superposition. This allows us to apply the Hadamard gate again to convert the qubit back into its original state of being 0 or 1 respectively. This can be seen in the table below:

Qubit	Hadamard Gate	Result
$\begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}$	$H \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}$	$\begin{pmatrix} 1 \\ 0 \end{pmatrix}$
$\begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}$	$H \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}$	$\begin{pmatrix} 0 \\ 1 \end{pmatrix}$

Table 6.4: Hadamard gate on qubits

This is particularly useful in quantum computing as it allows us to create a superposition at the start of a quantum circuit, and then convert the output back to a classical bit without measuring the qubit.

6.4.2. Pauli-X Gate (X):

The Pauli-X gate is a quantum gate that flips the state of a qubit, analogous to a classical NOT gate. It is defined as:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (6.6)$$

In a quantum circuit, the Pauli-X gate is often represented as:

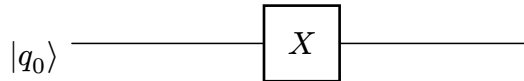


Figure 6.1: Pauli-X Gate

6.4.3. CNOT Gate (CX):

The CNOT gate is a two-qubit gate that flips the state of a target qubit if the control qubit is in the $|1\rangle$ state. It is defined as:

$$CX = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad (6.7)$$

In a quantum circuit, the CNOT gate is often represented as:

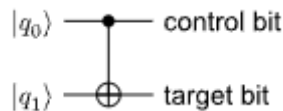


Figure 6.2: CNOT Gate

In terms of mathematics, when we apply the CNOT gate to two qubits, it looks like this:

$$CX (|q_0\rangle \otimes |q_1\rangle) \quad (6.8)$$

where q_0 is the control qubit and q_1 is the target qubit. The CNOT gate flips the state of q_1 if q_0 is in the $|1\rangle$ state. This is what we use to entangle qubits in quantum circuits, as it creates a correlation between the states of the two qubits.

6.5. Quantum Gate Matrices

Quantum gates can be represented as matrices that act on the state vectors of qubits. The state of a qubit can be represented as a vector in a two-dimensional complex vector space, and quantum gates are linear transformations that operate on these vectors. Here are some common quantum gate matrices:

Gate	Matrix	Description
I	$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ (6.9)	Identity Gate
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (6.10)	Pauli-X Gate
H	$\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix}$ (6.11)	Hadamard Gate
CX	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$ (6.12)	CNOT Gate

6.6. Describe how qubits are constructed and how they compare to classical bits

A qubit is the fundamental unit of quantum information, analogous to a classical bit. However, unlike classical bits, which can only be in one of two states (0 or 1), qubits can exist in a superposition of both states simultaneously. This means that a qubit can be in a state represented by a linear combination of $|0\rangle$ and $|1\rangle$, such as:

$$|q\rangle = \alpha|0\rangle + \beta|1\rangle \quad (6.13)$$

where α and β are complex numbers that represent the probability amplitudes of the qubit being in the $|0\rangle$ and $|1\rangle$ states, respectively. The probabilities of measuring the qubit in either state are given by $|\alpha|^2$ and $|\beta|^2$, which must satisfy the normalization condition $|\alpha|^2 + |\beta|^2 = 1$.

Qubits are generated by passing a classical bit through a quantum gate, such as the Hadamard gate, which creates a superposition of the $|0\rangle$ and $|1\rangle$ states. When we measure a qubit, the quantum waveform collapses to one of the two states with probabilities determined by the amplitudes. This is a fundamental difference from classical bits, which do not exhibit superposition or probabilistic behaviour.

Qubits can do anything that a classical bit can do, as they are a superset of the classical bits. However, they can also perform operations that classical bits cannot, such as superposition and entanglement. This allows quantum computers to perform certain computations (such as the Deutsch-Oracle problem) exponentially faster than classical computers.

6.7. What are the 4 main gates for a 2 qubit system?

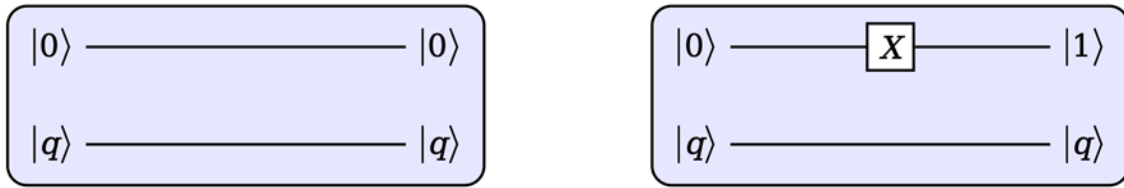


Figure 6.3: set 0 and set 1 gates

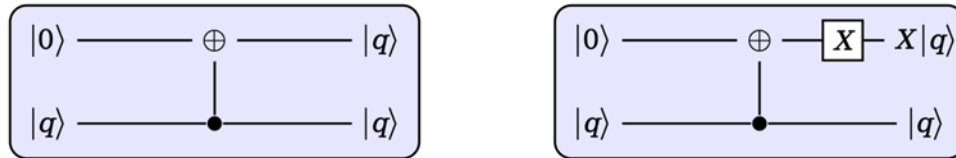


Figure 6.4: identity and negation gates

Note that this question is a bit weird, as even though it asks for a 2 qubit system, the top bit is the output bit, while in the bottom bit is the input bit. So really, we are only manipulating the top bit. But as all quantum circuits must be reversible, we have to set the circuits up like this.

6.8. Why is a Quantum Computer Better Than Conventional Computers in the context of the Deutsch Oracle Problem?

Deutsch oracle problem: Let's say we have a black box function (one which we cannot see). So:

$$f : \{\{0, 1\}\}^n \rightarrow \{0, 1\} \quad (6.14)$$

The BB takes in as input 0s and 1s and returns either a 0 or 1 depending on input. We want to know if the function of the BB is constant or balanced. If it is balanced, it outputs an even number of zeros or ones. If it is constant, it outputs either all zeros or all ones.

Function	F(0)	F(1)	type
set 0	0	0	constant
set 1	1	1	constant
Identity	0	1	balanced
Negation	1	0	balanced

Table 6.6: Examples of functions for the Deutsch oracle problem

If we had n bits, a classical computer would have to evaluate the function at least $2^{n-1} + 1$ times to determine if the function is constant or balanced. This is because it would have to evaluate the function at least once for each possible input combination of 0s and 1s, which is 2^n combinations. However, since we only need to determine if the function is constant or balanced, we can stop evaluating once we have seen enough inputs to make that determination.

A quantum computer, on the other hand, can evaluate the function in just one query using superposition and interference. It can create a superposition of all possible inputs and evaluate the function on all of them simultaneously. This allows it to determine if the function is constant or balanced in just one query. This is an exponential speedup! In terms of time complexity its $O(1)$, while for a classical computer it is $O(2^x)$. The quantum circuit that implements this is shown below:

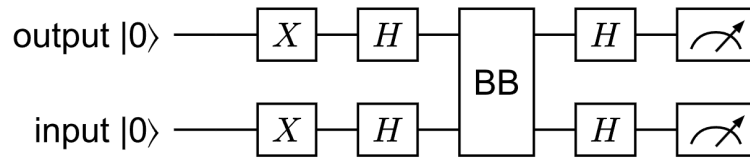


Figure 6.5: Quantum Circuit for the Deutsch-Jozsa Algorithm

This speed-up is possible due to two uniquely quantum mechanical phenomena: superposition and interference.

1. Superposition allows a quantum system to exist in multiple states simultaneously. In the Deutsch-Jozsa algorithm, the input qubits are placed in a superposition of all possible inputs. This means the quantum oracle (which implements the function $f(x)$) is effectively evaluated for all 2^n input values in parallel, not just one at a time.
2. Quantum parallelism alone isn't enough, however. The power of the algorithm comes from quantum interference, which enables the constructive and destructive combination of quantum states based on their phases. After the oracle is applied, the system undergoes a transformation (via Hadamard gates) that causes interference between the amplitudes of the various basis states. By using interference between the states of the qubits, the quantum computer can obtain information about the nature of the BB function with just one query. (Important distinction: quantum interference involves the interaction of a particle's wave functions, while entanglement links the quantum states of two or more particles).
3. The resulting interference pattern depends on whether the function is constant or balanced. Thus, a single measurement after the interference step reveals the global property of the function with 100% certainty.

The more rigorous version of this is the Deutsch-Jozsa algorithm, which generalises the Deutsch oracle problem to multiple qubits.

Quantum computers are not better than classical computers in every single situation. As, not every problem can take advantage of quantum parallelism. However, they are at least a good or substantially better than classical computers. The only downside is that matching a classical computer in terms of memory is extraordinarily difficult.

6.9. Practical Limitations

There are many practical limitations to quantum computers, such as:

1. Quantum decoherence: Quantum states are fragile and can easily be disturbed by their environment (even small vibrations), leading to loss of information. This is known as decoherence, and it is a major challenge in building practical quantum computers. It is

why current quantum computers look like chandeliers, as they need to be kept as stable as possible to prevent decoherence.

2. Scalability: Building large-scale quantum computers with many qubits is challenging due to the difficulty of maintaining coherence and controlling interactions between qubits.
3. Error rates: Quantum gates are not perfect, and errors can accumulate during computations. Error correction techniques are needed to mitigate this issue, but they require additional qubits and resources.
4. Noise: Quantum systems are susceptible to noise, which can introduce errors in computations. Noise can come from various sources, such as thermal fluctuations, electromagnetic interference, and imperfections in the hardware.
5. Complexity of algorithms: Designing efficient quantum algorithms for practical problems is still an active area of research. While some problems have known quantum algorithms that outperform classical ones, many practical problems remain unsolved.
6. Memory limitations: Quantum computers require specialized memory systems to store quantum states, which can be challenging to implement at scale. The memory must be able to maintain coherence and support fast read/write operations. Classical computer systems as of right now are still significantly better than quantum computers in terms of memory capacity.